

Obsah

1	Úvod	3
2	Co je kombinatorická hra	4
3	Kombinatorické hry	6
3.1	Hackenbush	6
3.2	Hotpotch	7
3.3	Nim	7
3.4	Neformální přiřazení čísel pozicím v Hackenbushi	8
4	Krátké hry	10
4.1	Algebra her	13
4.2	Infinitezimální hodnoty	18
5	Dyadická čísla	22
5.1	Definice dyadického čísla	22
6	Návrh a implementace pomocí dyadických čísel	25
6.1	Datové reprezentace grafu Hackenbushe	25
6.1.1	Pravidlo jednoduchosti - nejjednodušší dyadické číslo mezi dyadickými čísly	26
7	Návrh a implementace pomocí krátkých her	30
7.1	Od surové hry ke krátké hře	30
7.2	Výsledek hry	31
7.3	Explicitní rekurzivní převod surové hry	31
7.4	Memoizace a hashovací tabulky	33
7.5	Kanonizace a její důsledky	34
7.6	Rozdělení Hotpotche na nezávislé komponenty	36
7.7	Porovnávání her: $G \geq H$	37
8	Zobecnění řešice krátkých her pro ostatní hry	38
8.1	Obecnější převod surové hry na hru krátkou	39
8.2	Domineering	42
8.3	Struktura souborů	43
9	Porovnání přístupů a další rozšíření	47
10	Závěr	49
	Literatura	50
A	Návod k instalaci a spuštění	51

Seznam obrázků

3.1	Příklad triviální pozice v Hackenbushu.	6
3.2	Zelená hrana v Hotpotchi, obsahující jednu zelenou hranu, která vychází z kořene	7
3.3	Pozice v Nimu s hromádkami velikostí 6, 4 a 8 znázorněná jako Green Hackenbush. Každá hromádka odpovídá jednomu řetězu zelených hran nad zemí.	7
3.4	Tři základní různé pozice v Hackenbushu	8
3.5	Obrázek znázorňuje proces zjišťování hodnoty pozice na obrázku 3.5a postupným přidáváním dalších známých hodnot. Podobný postup lze najít v [1, str. 4–5]. . .	9
4.1	Ukázka dvou pozic kde každá z nich porušuje alespoň jednu omezující podmínku krátkých her.	11
4.2	Rozklad pozice AB na nezávislé komponenty A a B , tedy $AB = A + B$	13
4.3	Vizuální ukázka převodu surové kombinatorické hry na krátkou hru	15
4.4	Pozice modrého na řadě, obsahující dominovaný tah s indexem 0 dominovaným tahem s indexem 1	16
4.5	Pozice modrého na řadě, obsahující reverzibilní tah s indexem 1, reverzibilní přes tah s indexem 0	16
4.6	Ukázka rozkladu hry s jednou zelenou hranou vycházející ze země. Modrá šipka značí tah levého hráče do nulové hry vlevo dole, červená šipka analogicky značí tah pravého hráče do nulové hry vpravo dole.	18
4.7	Pozice s hodnotou $*$ – 1.	19
4.8	Ukázka porovnání $*$ s kladnými dyadickými čísly.	19
4.9	Ukázka porovnání $*$ se zápornými dyadickými čísly.	19
4.10	Pozice a má fuzzy hodnotu — vyhraje vždy hráč na řadě. Ale součet se stejnou pozicí b je výhra pro levého.	20
4.11	Pozice s hodnotou $*$ znázorněna ve hře Hotpotch.	20
4.12	Pozice s hodnotami $\uparrow$ (a) a $\downarrow$ (b) znázorněny ve hře Hotpotch.	21
5.1	Stromová reprezentace dyadických čísel, kde vodorovné čárkované čáry vyznačují jednotlivé dny narození	23
6.1	Datová reprezentace Hackenbushe a její vizuální ukázka s 5 vrcholy a 4 hranami.	25
8.1	Ukázka nepravidelné pozice ve hře Domineering.	42
8.2	Stromová struktura adresářů a souborů se zeleně vyznačenými částmi, se kterými nově implementovaná kombinatorická hra interaguje.	44
8.3	Ukázka grafického rozhraní Hotpotche s vysvětlivkami daných rozšíření.	45
8.4	Ukázka grafického rozhraní kalkulačky krátkých her.	46

Kapitola 1

Úvod

Kombinatorická teorie her (CGT) je matematická disciplína zabývající se analýzou dvouosobových her s dokonalou informací, ve kterých na výsledku závisí pouze strategie hráčů, nikoliv náhoda.

V minulosti se analýza her skládala nejčastěji z nejednotné studie různorodých nesouvisících her. V této době ani nebylo možné studovat komplexnější hry a tato disciplína nebyla výrazně posunuta dopředu. Toto se změnilo kolem roku 1930, kdy R. P. Sprague a P. M. Grundy přinesli formálnější popis her. Z práce těchto dvou autorů je významná Spragueova-Grundyho věta, protože ukazuje, že širokou třídu her lze převést na hru Nim, a tím je možné studovat jednotným způsobem. Následně, v roce 1976, John H. Conway vydal knihu *On Numbers and Games*, kde poprvé formálně popsal práci s hrami tak, jak ji známe nyní. Conway ukázal, že nad strukturou možných tahů prvního a druhého hráče lze formálně definovat uspořádání, sčítání a další aritmetické operace pro analýzu her. Tato práce obsahuje pouze velmi malou část možné analýzy, protože se stále jedná o relativně čerstvé téma.

V kapitole 5 je formálně popsáno, že některé kombinatorické hry lze vyhodnocovat pouze jako dyadická čísla. Jedna z takových her je Hackenbush, který je použit pro představení těchto čísel, a následně v kapitole 6 je popsán způsob, jak takovou hru v Hackenbushu převést na dyadické číslo.

Druhá metoda využívá aparátu tzv. krátkých her. Jedná se právě o hodnoty vyjádřené možnými tahy prvního a druhého hráče. Tato forma hry je obecnější a je využita ve hře Hotpotch, která vyžaduje tuto obecnější formu pro vyhodnocování hodnoty. Tento způsob, vysvětlený v kapitole 8, lze následně použít pro vyhodnocování dalších libovolných kombinatorických her.

Kapitola 2

Co je kombinatorická hra

Neexistuje jediná správná kolekce pravidel kombinatorické hry a v textech lze najít různé varianty. Pro tuto práci stačí pravidla vycházející z [5, 10]. Kombinatorická hra je hra, která splňuje následující pravidla.

1. Existují dva hráči, levý hráč a pravý hráč, zkráceně pouze levý a pravý.
2. Levý a pravý hráč se střídají v tazích.
3. Oba hráči mají všechny informace, respektive úplnou informaci (complete information).
4. Žádný tah není ovlivněn náhodnými jevy, jako jsou hod kostkou nebo míchání karet.
5. Podle normální konvence hráč, který již nemůže provést tah, prohrává.
6. Oba hráči jsou racionální.

V každé kapitole bude upozorněno na pravidla, která jsou porušena. Tyto však zůstanou vždy.

Mezi pravidla, která budou v některých kapitolách porušena, patří

7. hra obsahuje konečné množství pozic, ve kterých se může nacházet.
8. hru musí jít vždy ukončit, protože některý z hráčů už nemůže vykonat tah. Toto je také nazýváno koncovou podmínkou (ending condition).

Klasickým příkladem kombinatorické hry je Nim 3.3. Jedná se o hru s hromádkami žetonů, kde každý hráč odstraní libovolný počet žetonů z jedné hromádky. Výsledek libovolné pozice lze určit pomocí nim-součtu, jak dokázal Bouton v roce 1902 [2].

Každou pozici v Nimu lze triviálně převést na pozici v Hotpotchi, kde je každá hrana zelená. Vytvořením řešiče (solveru) jakékoliv pozice v Hotpotchi, představené v sekci 3.2, tedy vytvořím i řešič Nimu. Na druhou stranu, pokud lze ověřit, že daná pozice v Hotpotchi odpovídá pozici v Nimu, lze použít efektivní pravidlo nim-součtu pro řešení takové pozice.

Cíle kombinatorických her

Kombinatorická teorie her (CGT) se zabývá hned několika cíli. Pro mě bude nejdůležitější, při zadané pozici, zjistit výsledek hry, tedy intuitivně odpovědět na otázku, kdo vyhraje. Tento cíl rovnou rozšířím o otázku, "jak moc je pozice výherní".

Detailněji se tento text zabývá těmito cíli:

- Najít přesný výsledek pozice zadané hry, nejčastěji se jedná o algebraický výraz.
- Zjistit, do jaké třídy patří nějaká pozice hry.
- Porozumět hře z pohledu kombinatorických her a využít její vlastnosti.

Třída výsledků

Přirozenou otázkou u každé kombinatorické hry je, který hráč vyhraje při optimální hře obou stran. Pozice hry lze rozdělit do *tříd výsledků* (outcome classes). Obecně existují čtyři třídy:

1. Pozice, kde vyhraje první hráč ($\mathcal{N}$, z angl. *next*).
2. Pozice, kde vyhraje druhý hráč ($\mathcal{P}$, z angl. *previous*).

U symetrických her, jako je Nim, existují pouze dvě třídy. U asymetrických her uvažujeme další dvě třídy:

3. Pozice, kde vyhraje levý hráč bez ohledu na pořadí ($\mathcal{L}$).
4. Pozice, kde vyhraje pravý hráč bez ohledu na pořadí ($\mathcal{R}$).

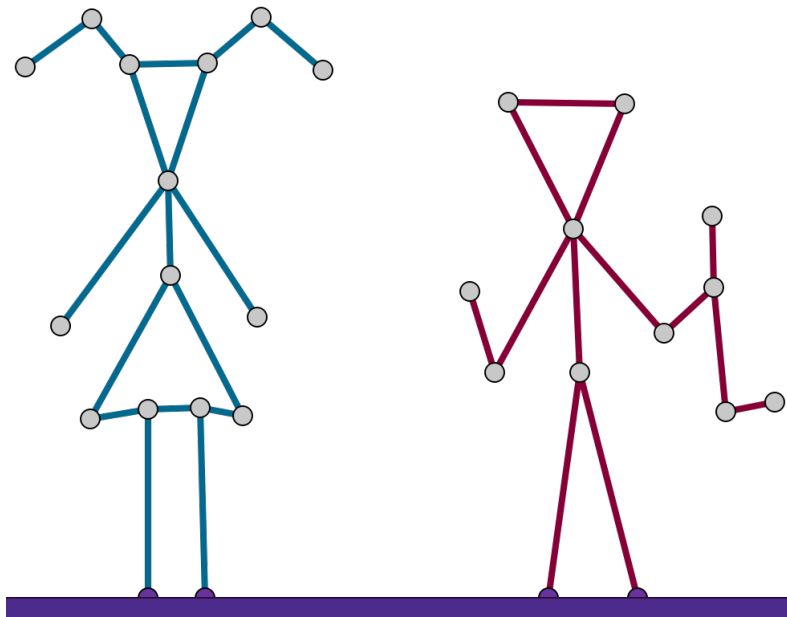
Tradičně říkáme, že hra je *vyřešena*, pokud existuje efektivní pravidlo pro určení třídy výsledku libovolné pozice [10, str. 4]. V této práci a v rámci implementace tímto pojmem rozumíme převedení hry na její algebraickou formu, pro kterou existuje efektivní pravidlo pro určení třídy výsledku.

Kapitola 3

Kombinatorické hry

3.1 Hackenbush

Hackenbush je kombinatorická hra, kde se střídají dva hráči, levý a pravý, respektive modrý a červený. Levý hráč táhne tak, že odstraní libovolnou modrou hranu spolu se všemi hranami, které již nejsou připojeny k zemi. Země bude na obrázcích značena nejčastěji fialovým obdelníkem. Pravý hráč táhne obdobně tak, že odstraní červenou hranu [1, str. 2]. Hackenbush začíná na startovní pozici, ve které se nachází modré a červené hrany. Každá hrana je buď spojena s jinou hranou, nebo se zemí. Podle normální konvence (normal play convention) prohrává hráč, který již nemůže zahrát tah. Existuje i varianta misère, kde vyhrává hráč, který již nemá tah.



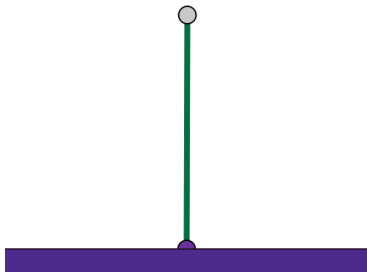
Obrázek 3.1: Příklad triviální pozice v Hackenbushi.

Na pozici znázorněnou obrázkem 3.1 se nachází 2 rozdělené části, které se nijak neovlivňují, což znamená, že žádný tah protějšího hráče neovlivní výběr nadcházejícího tahu. Pochopitelně lze dojít k úvaze, že když se levý (respektive modrý) a pravý (respektive červený) střídají v tazích, tak vyhraje hráč s více tahy, v tomto případě modrý, který má 17 tahů, zatímco červený má 13 tahů. V jistém smyslu by se dalo této pozici přiřadit číslo $17 - 13 = 4$.

3.2 Hotpotch

Hotpotch je varianta klasického Hackenbushe. Hra zůstává stejná, ale je přidán nový, zelený typ hrany. Zelenou hranu může zničit jakákoliv strana, tedy levá (resp. modrá) nebo pravá (resp. červená). Modré hrany ale pořád může ničit jen levý a červené hrany pořád jen pravý [5, str. 188]. Takové tahy, které mají na výběr obě strany, vytvářejí hodnoty, které nelze vyjádřit jako dyadické číslo. Na této hře bude představen obecný aparát krátkých her.

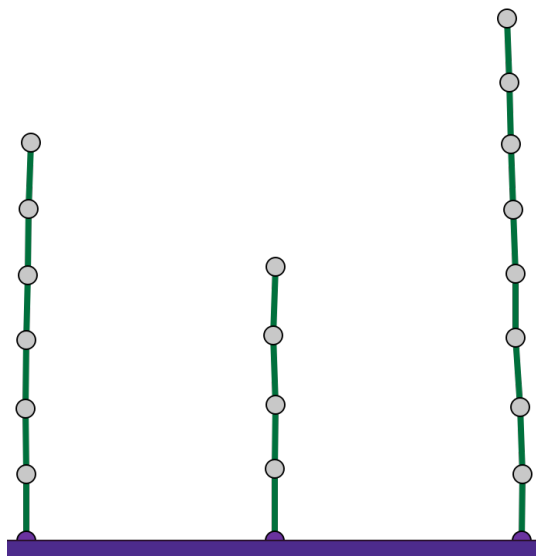
Možný příklad pozice v Hotpotchi je pouze jedna zelená hrana vycházející z kořene (země).



Obrázek 3.2: Zelená hrana v Hotpotchi, obsahující jednu zelenou hranu, která vychází z kořene

3.3 Nim

Nim je kombinatorická hra, kde se střídají dva hráči, levý a pravý. Hra začíná na startovní pozici, ve které se nachází několik hromádek žetonů. Každý hráč ve svém tahu odstraní libovolný počet žetonů (nejméně však jeden) z právě jedné hromádky. Podle normální konvence prohrává hráč, který již nemůže zahrát tah, tedy hráč, na jehož tahu jsou všechny hromádky prázdné.



Obrázek 3.3: Pozice v Nimu s hromádkami velikostí 6, 4 a 8 znázorněná jako Green Hackenbush. Každá hromádka odpovídá jednomu řetězu zelených hran nad zemí.

Pozici Nim na obrázku 3.3 lze přirozeně znázornit jako Hotpotch pozici [10, str. 180], kde každá hromádka o k žetonech odpovídá řetězu k zelených hran nad zemí. Jejich ekvivalence by šla dokázat například pomocí Spragueova-Grundyho věty. Odstranění žetonů z hromádky pak odpovídá přeříznutí hrany v řetězu.

Řešení Nim pozice

Výsledek libovolné Nim pozice lze určit pomocí *nim-součtu*. Necht G je pozice hry Nim obsahující hromádky o velikostech $a_1, a_2, \dots, a_n$. Potom Nim-hodnota této pozice je

$$a_1 \oplus a_2 \oplus \dots \oplus a_n,$$

kde $\oplus$ značí bitovou operaci XOR.

Ověříme si základní prázdnou Nim pozici, kde $a_1 = a_2 = \dots = a_n = 0$, potom Nim-hodnota této pozice je 0. V tomto případě je to výhra pro 2. hráče.

Věta 3.1 (Boutonova věta). *Necht G je pozice hry Nim. Pokud je G nulová pozice, pak každý tah z G vede do nenulové pozice. Pokud G není nulová pozice, pak existuje tah z G do nulové pozice.*

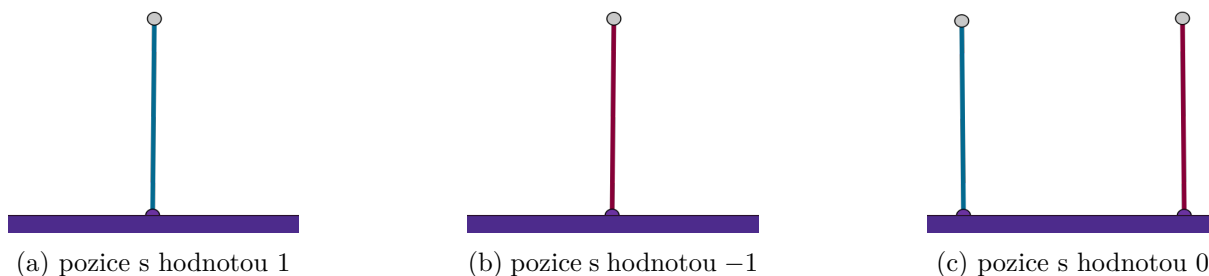
Počtem zbývajících žetonů lze induktivně dokázat, že hráč držící invariant předání nulové pozice soupeři nakonec předá soupeři nulovou pozici (všechny hromádky jsou prázdné), tedy prohru pro hráče na tahu.

Detekcí takové pozice v Hotpotchi by bylo možné optimalizovat nalezení výherní strategie.

3.4 Neformální přiřazení čísel pozicím v Hackenbushi

Conway ve svém dílu On Numbers and Games [5] předvedl matematickou formu her poměrně brzy. Mně dává větší smysl si prvně vytvořit intuici pro to, co by se mělo očekávat. Proto chci v této sekci neformálně představit přiřazování čísel (resp. hodnot) kombinatorickým hrám.

Jako příklad vezmu kombinatorickou hru Hackenbush a pokusím se zjistit, kdo v nějaké pozici vyhraje a přiřadit takové pozici smysluplné číslo.



Obrázek 3.4: Tři základní různé pozice v Hackenbushi

Na obrázku 3.4a se nachází pozice s jednou modrou hranou. Pokud začne levý (resp. modrý), zničí modrou hranu, a pravý (resp. červený) nemůže vykonat žádný tah, takže podle normální konvence vyhrál levý. Pokud začne pravý, nemá žádný možný tah a podle normální konvence znovu prohraje pravý. Jedná se tedy o pozici typu $\mathcal{L}$. Podobným argumentem lze dojít k tomu, že pozice s jednou červenou hranou na obrázku 3.4b je pozice typu $\mathcal{R}$.

Jak je to s pozicí na obrázku 3.4c? Tato pozice je dostatečně jednoduchá na to, aby bylo možné projít všechny možné tahy z pohledu levého i pravého, respektive modrého i červeného hráče. V takové pozici vyhraje druhý hráč na řadě, jedná se tedy o pozici typu $\mathcal{P}$. Neformálně řečeno, taková pozice je vyvážená.

Cílem tohoto souhrnu je ukázat, že v Hackenbushi existují pozice, které může jednoznačně vyhrát levý nebo pravý hráč, ale ne oba.

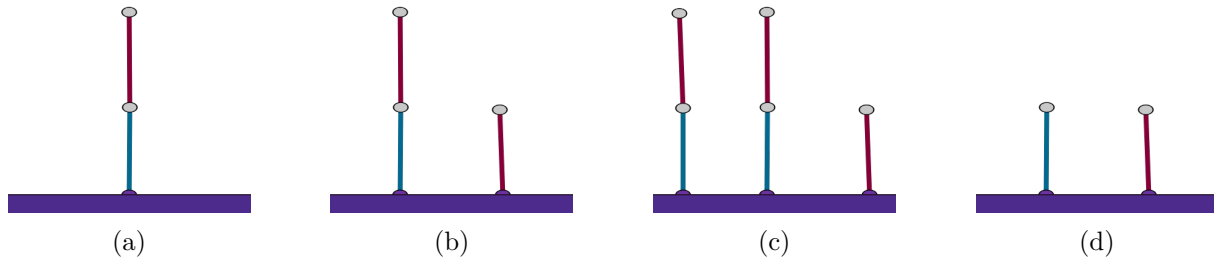
Nejsmysluplnější číslo pozice s jednou modrou hranou je 1 nebo -1 . Jaká hodnota z nich to je, je arbitrární, ale podle konvence má jedna modrá hrana číslo 1. Když se k této modré hraně přidá červená hrana (obrázek 3.4c), hra se přesune z jednoznačné výhry pro levého na výhru pro druhého hráče. Kdyby se ke stejné hře přidala další červená hrana vycházející ze země, potom

je taková hra jednoznačná výhra pro pravého hráče. Šlo by říct, že někde mezi jednoznačnou výhrou pro levého a jednoznačnou výhrou pro pravého se nachází neutrální hodnota, která není ve prospěch žádného hráče, a vyhraje druhý hráč na řadě. Podle konvence se takové pozici přiřadí číslo 0.

Poznámka. V Hackenbushu neexistuje pozice, kde vyhraje vždy první hráč na řadě. Takové pozice existují v Hotpotchi.

Poznámka. Co je to *číslo*, je popsáno v kapitole 5. Aktuální sekce se zabývá pouze neformálním představením.

Pozice s číselnou hodnotou poloviny tahu



Obrázek 3.5: Obrázek znázorňuje proces zjišťování hodnoty pozice na obrázku 3.5a postupným přidáváním dalších známých hodnot. Podobný postup lze najít v [1, str. 4–5].

Klasickou metodou zjištění čísla bez aparátu krátkých her (více v kapitole 4) je přidat k neznámé pozici již známou pozici. Vezmu příklad 3.5a, lze se znovu přesvědčit, že takovou hru vyhraje jednoznačně levý (resp. modrý). Přidáním jedné červené hrany, jako na obrázku 3.5b, naopak vyhraje jednoznačně pravý hráč. Pozice 3.5a tedy nemá dostatečnou hodnotu na vynulování výsledné pozice.

Lze se ale přesvědčit, že pozice na obrázku 3.5c je už nulová pozice. Pokud levý začíná, tak zničí modrou hranu, pravý zničí červenou hranu na modré hraně a dostane se na známou nulovou pozici, kde vyhraje druhý hráč (obrázek 3.5d), tedy pravý hráč. Pokud začne pravý, zničí červenou hranu připojenou k vrcholu modré hrany, levý poté zničí modrou hranu s červenou hranou na ní, která dostane pozici do známé nulové pozice (obrázek 3.5d), tedy vyhraje levý. Takže dvě tyto neznámé pozice nulují pozici s hodnotou -1 . Dalo by se říct, že pozice na obrázku 3.5a má tedy hodnotu $\frac{1}{2}$? Pokud přijmeme předpoklad, že pozice se můžou takovým způsobem sčítat (můžou), tak ano, taková pozice má hodnotu $\frac{1}{2}$.

Konvence je tedy taková, levý (resp. modrý) chce hodnotu pozice maximalizovat, naopak pravý (resp. červený) chce hodnotu pozice minimalizovat.

Metoda v této sekci je ale nedostatečná. Je potřeba zavést formální aparát, který každé pozici přiřadí přesnou hodnotu a umožní s těmito hodnotami dále pracovat. Tímto aparátem jsou krátké hry.

Kapitola 4

Krátké hry

V předchozí kapitole jsem hodnotám pozic přiřazoval čísla experimentálně - přidáváním známých pozic a pozorováním výsledku. Takový postup ale nelze použít pro důkazy ani pro implementaci. Je potřeba formální model, který každé pozici přiřadí hodnotu přímo z její struktury. Tímto modelem jsou krátké hry.

Krátké hry dokážou obecně vyjádřit hodnoty jiného typu, než jsou celá a racionální čísla představená v minulé sekci 3.4. I když taková čísla lze představit jednoduše i neformálně, dává větší smysl dříve představit obecnější formu — krátké hry.

Z krátkých her lze dále sestavit celá a dyadická čísla. Co je to číslo, bude představeno právě v kapitole 5.

Definice krátké hry

Místo popisu pomocí specifických pravidel lze pro matematickou analýzu hru popsat abstraktně, a to výhradně skrze tahy, které mají hráči k dispozici.

Definice 4.1 (Krátká hra). *Následující definice je převzata z [10, str. 53–54]. Krátká hra G je uspořádaná dvojice $(\mathcal{G}^L, \mathcal{G}^R)$, kde $\mathcal{G}^L$ a $\mathcal{G}^R$ jsou množiny „jednodušších“ krátkých her, **levé** a **pravé** možnosti hry G .*

Často budu označovat možnosti levého ve hře G jako G^L a možnosti pravého jako G^R . Zjednodušeně lze psát

$$G = \{G^L \mid G^R\}$$

Nebo v rozepsaném tvaru jako

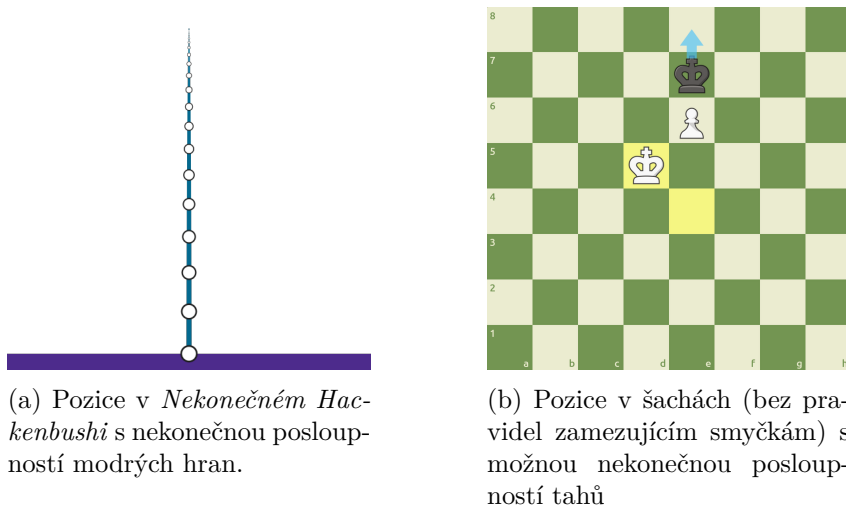
$$G = \{G^{L_1}, G^{L_2}, \dots \mid G^{R_1}, G^{R_2}, \dots\}$$

*Základním případem rekurze je **nulová** hra $0 = \{\mid\}$, taktéž nazývaná **prázdná hra** [5, 10].*

Jedná se o zjednodušenou definici, ze které nemusí být snadné poznat, že krátká hra má vždy konečný počet prvků. Proto explicitně představuji další požadavky krátkých her.

1. *Konečnost*: krátká hra má pouze konečně mnoho podpozic (stavů, do kterých lze dospět).
2. *Bezsmýškovost* (loopfree): krátká hra nemůže pokračovat donekonečna, neexistuje nekonečná posloupnost tahů.

Hodnoty krátkých her tvoří grupu $\mathbb{G}$ vzhledem k operaci součtu her, kde neutrálním prvkem je $0 = \{\mid\}$ [10, str. 56].



Obrázek 4.1: Ukázka dvou pozic kde každá z nich porušuje alespoň jednu omezující podmínku krátkých her.

Obrázek 4.1a ukazuje pozici v rozšířené verzi Hackenbushe, kde pozice může mít nekonečně mnoho možností. Tato pozice obsahuje nekonečně mnoho modrých hran, každá vycházející z té předešlé. Taková pozice není krátká hra, protože porušuje podmínku *konečnosti*. Neporušuje ale podmínku bezsmyčkovosti, protože jakýkoliv tah zničí zbývajících nekonečně mnoho možností. Jedná se tedy o pozici, která porušuje pravidlo 7

Obrázek 4.1b ukazuje pozici v šachách bez pravidla *padesáti tahů*. Pokud by se bílý hráč pokusil vyhrát takovou pozici, černý hráč by při optimální strategii dokázal donekonečna cyklit hru. Taková pozice tedy porušuje pravidlo *bezsmýčkovosti*. Jedná se tedy o pozici, která porušuje pravidlo 8

Poznámka. Šachy navíc nejsou kombinatorickou hrou, protože existují pozice, které můžou skončit remízou, a porušují tím podmínku 5. Důvod, proč vůbec dává smysl mluvit o šachách, je, že některé metody lze aplikovat i na hry s možnou remízou, například práce *Noama Elkiese* [6].

Hackenbush, Hotpotch a Nim tyto vlastnosti splňují.

Definice je rekurzivní a krátká hra je definována skrze jednodušší hry. Rekurse ale vždy skončí u nulové hry 0, protože krátké hry jsou konečné a bezsmyčkové.

Základní terminologie

V textu budu často pracovat s krátkou hrou ve tvaru

$$G = \left\{ G^L \mid G^R \right\},$$

kde G^L označuje množinu levých možností a G^R množinu pravých možností. Zápisem $|G^L|$ myslím počet prvků množiny G^L , tedy počet pozic dosažitelných jedním tahem levého hráče. Analogicky $|G^R|$ značí počet pozic dosažitelných jedním tahem pravého hráče.

Pojmy *hra* a *pozice* budu používat podle kontextu. Slovem *hra* budu nejčastěji označovat krátkou hrou, která je v daném místě hlavním zkoumaným objektem. Slovem *pozice* budu často označovat krátkou hrou [5, odst. 6, kap. 7], která v daném kontextu vystupuje jako prvek množiny možností jiné hry [9, str. 2]. Například každý prvek množiny G^L je sám krátkou hrou (pokud je G krátká hra), ale vzhledem ke hře G jej budu označovat jako levou pozici nebo levou možnost.

Pojmy *hra* (resp. krátká hra) a *pozice* jsou zaměnitelné. Důvodem je, že každá pozice sama určuje krátkou hrou, která začíná právě v této pozici. Jejich odlišení je pouze stylistickým prvkem.

Prvky množiny levých možností budu značit například

$$G^{L_1}, G^{L_2}, \dots$$

nebo obecně G^{L_i} . Analogicky budu prvky pravých možností značit

$$G^{R_1}, G^{R_2}, \dots$$

nebo obecně G^{R_i} .

Pokud budu chtít označit pozici vzniklou pravým tahem z levé možnosti G^{L_i} , formálně by šlo psát například:

$$\left(G^{L_i}\right)^{R_j}.$$

Pro přehlednost ale budu používat zkrácené značení

$$G^{L_i R_j}.$$

Tento zápis znamená, že se ze hry G nejprve provede i -tý levý tah, a poté se z výsledné pozice provede j -tý pravý tah. Podobně zápis $G^{L_i R_j L_k}$ označuje pozici dosaženou posloupností levého, pravého a znovu levého tahu.

Conway ve svém díle *On Numbers and Games* [5], volně zaměňuje pojmy *hra* a *krátká hra* [5, str. 71–72]. Existují i dlouhé hry, kde každá množina pozic může mít nekonečně mnoho prvků. O takových hrách se zde ale nebude mluvit. Budu často explicitně uvádět, že se jedná o krátkou hru; pokud však v daném kontextu dává větší smysl použít pouze pojem *hra*, myslí se tím krátká hra.

Pod pojmem *vyřešit hru* se zde myslí převést surovou hru na formu krátké hry.

Dále předpokládám, že všechny zmíněné definice jsou induktivní. Pokud tedy například něco říkám o objektu x , předpokládám, že zmíněná vlastnost je už aplikovaná na pár (x^L, x^R) . Stejnou metodu používá i Conway v [5], kterou vysvětluje na straně 16.

4.1 Algebra her

Definice 4.2 (Negace z [5, str. 5]). Negace hry G je hra $-G$ definovaná rekurzivně jako

$$-G = \left\{ -G^R \mid -G^L \right\}.$$

Hra $-G$ je tatáž hra s prohozenými rolemi hráčů, respektive prohozenými barvami hran.

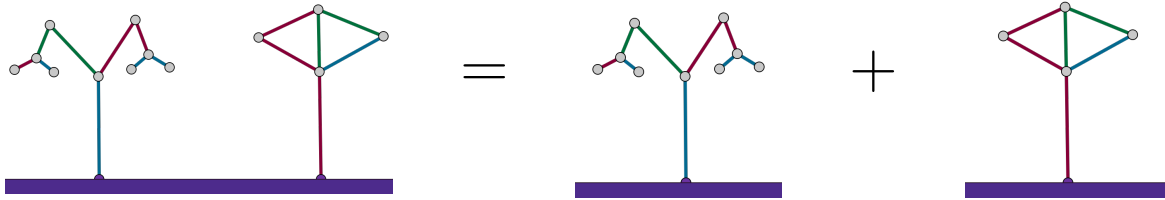
Pro každou krátkou hru G platí $G + (-G) = 0$: druhý hráč vždy vyhraje $G + (-G)$ kopírováním každého tahu soupeře na druhé komponentě [1, str. 33–34].

Hry lze sčítat. Pokud se hraje více nezávislých her najednou (například více komponent Hackenbushe), v každém tahu se vybere jedna z nich a vykoná se v ní tah.

Definice 4.3 (Součet z [5, str. 5]). Necht G a H jsou krátké hry. Jejich disjunktivní součet $G + H$ je definován rekurzivně jako

$$G + H = \left\{ G^L + H, G + H^L \mid G^R + H, G + H^R \right\},$$

kde $G^L + H$ značí množinu všech her tvaru $G^{L_i} + H$ a obdobně pro ostatní členy.



Obrázek 4.2: Rozklad pozice AB na nezávislé komponenty A a B , tedy $AB = A + B$.

Je dobré zmínit, že tento postup rozkladu na hry, které se navzájem neovlivňují, funguje obecně i pro rozdílné hry. Je tedy možné například sečíst pozici v Hackenbushu s pozicí v nimu.

Všechny očekávané vlastnosti sčítání zůstávají, tedy především:

Věta 4.1 (Vlastnosti sčítání z [5, str. 17–18]). Pro libovolné krátké hry x , y a z platí:

1. $x + 0 \equiv x$,
2. $x + y \equiv y + x$,
3. $(x + y) + z \equiv x + (y + z)$,
4. $-(x + y) \equiv (-x) + (-y)$,
5. $-(-x) \equiv x$,
6. $x + (-x) = 0$.

Říkáme, že hry x a y jsou ekvivalentní, píšeme $x \equiv y$, pokud jsou ekvivalentní jejich levé a pravé množiny. To znamená, že každá levá možnost x^L je ekvivalentní s nějakou levou možností y^L , každá pravá možnost x^R je ekvivalentní s nějakou pravou možností y^R , a naopak [5, str. 15–16].

Poznámka. Řádek 6 je rovnost spíše než ekvivalence.

Hodnota hry

Definice 4.4 (Uspořádání, převzaté z [5]). $G \geq H$ právě tehdy, když pro každý pravý tah G^R z G platí $\neg(H \geq G^R)$ a pro každý levý tah H^L z H platí $\neg(H^L \geq G)$:

$$G \geq H \iff (\forall G^R : \neg(H \geq G^R)) \wedge (\forall H^L : \neg(H^L \geq G)).$$

Dvě hry mohou vypadat zcela jinak a přesto mít stejnou *hodnotu* – chovat se stejně v každém kontextu.

Definice 4.5 (Rovnost her, převzatá z [5]). Hry G a H jsou si rovny jako hodnoty, píšeme $G = H$, pokud $G \geq H$ a zároveň $H \geq G$.

Uspořádání $G \geq H$ vyjadřuje, že hra G je pro levého hráče alespoň tak dobrá jako H v jakémkoliv kontextu.

Důležitá poznámka je ta, že rovnost je definována jako relace. Zdánlivě různé definice tedy mohou určovat totéž číslo, a proto je třeba rozlišovat mezi tvarem $\{L \mid R\}$ čísla a číslem samotným.

Všechny definice jsou induktivní. Chceme-li tedy například rozhodnout, zda $x \leq y$, musíme uvažovat několik podobných otázek pro dvojice x^R, y a x, y^L . Tyto otázky jsou však jednodušší než původní problém. Na první pohled nemusí být zcela zřejmé, že tyto indukce nevyžadují žádný základní případ, protože nakonec jsme redukováni na otázky týkající se prvků prázdné množiny [5, str. 4–5].

Přiřazování krátkých her pozicím v Hotpotchi

Nyní, když je možné formálně sčítat a porovnávat krátké hry, je dobré ukázat, jak různé pozice v Hotpotchi vypadají jako krátké hry. Takový postup lze použít pro libovolnou kombinatorickou hru.

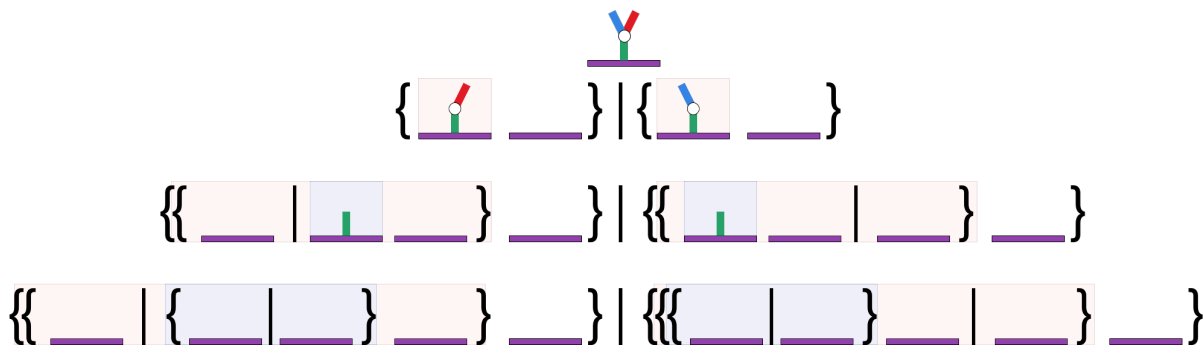
Krátká hra G má tvar $\{G^L \mid G^R\}$, kde prvky levé množiny označují pozice dostupné levému hráči a prvky pravé množiny označují pozice dostupné pravému hráči.

Nejjednodušší případ je nulová hra, kde ani jeden z hráčů nemá možný tah. Taková hra má hodnotu $\{ \mid \}$. Jak bylo popsáno v sekci 3.4, tato hodnota je rovna 0, tedy $0 = \{ \mid \}$. Říkáme, že taková pozice je nulová [5, str. 75].

Vezměme příklad 3.4a, kde se nachází pouze jedna modrá hrana. Pokud levý hráč tuto hranu zničí, dostane nulovou pozici; pravý hráč od začátku táhnout nemůže. Takovou pozici zapíšeme jako $\{0 \mid \}$. Toto je kanonická forma hodnoty 1. Analogicky $2 = \{1 \mid \}$, $3 = \{2 \mid \}$ a pro červené hrany $-1 = \{ \mid 0\}$, $-2 = \{ \mid -1\}$ atd. Tímto způsobem lze vytvořit všechna celá čísla.

Jak zapsat pozici na obrázku 3.5a s modrou hranou vycházející ze země a červenou hranou na modré hraně? Levý hráč zničí modrou hranu a získá nulovou pozici; pravý hráč zničí červenou hranu a získá pozici 1. Zápis je $\{0 \mid 1\}$.

Taková pozice se chová jako číslo $\frac{1}{2}$, protože jde o nejjednodušší hodnotu mezi 0 a 1 — tu s nejmenším dnem narození, jak je znázorněno na obrázku 5.1. Přesný smysl slova *nejjednodušší* a formální důvod, proč lze herní formy ztotožnit s dyadickými racionálními čísly, jsou vysvětleny v kapitole 5. Toto přiřazení beru jako motivované pozorování.



Obrázek 4.3: Vizuální ukázka převodu surové kombinatorické hry na krátkou hru

Stejný postup lze uplatnit i pro zelené hrany: zničení zelené hrany je dostupné oběma hráčům, takže příslušná pozice se objeví v množinách tahů obou hráčů. Hodnotu na obrázku 4.3 lze zapsat jako

$$\left\{ \{0 \mid \{0 \mid 0\}, 0\}, 0 \mid \{\{0 \mid 0\}, 0 \mid 0\}, 0 \right\}.$$

Po kanonizaci se tato hodnota zjednoduší na $\{0 \mid 0\}$. Pro takovou hru neexistuje číselná hodnota ve smyslu dyadických čísel (viz sekce o dyadických číslech 5.1). Přesvědčit se lze i tak, že téhož výsledku dosáhneme jedinou zelenou hranou vycházející ze země. Tyto speciální pozice jsou podrobněji probrány v sekci o infinitezimálních číslech 4.2.

Kanonická forma

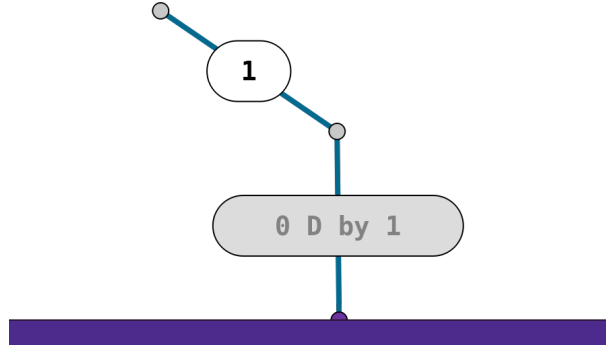
Například zápis $G = \{0, 0, 0, 0 \mid 2\}$ je platný zápis krátké hry, ale takový zápis *není* v kanonické formě. Krátkou hru lze převést do kanonické formy odstraněním dvou typů tahů: dominovaných a reverzibilních [10, str. 64–65]. Znovu upozorňuji čtenáře na to, že všechny definice jsou indukativní, tedy pokud nějaká vlastnost platí pro nějaký objekt x , potom předpokládám, že už platí například pro pár (x^L, x^R) .

Dominované tahy

Nechť levý hráč má v pozici G k dispozici tah do G^{L_i} a tah do G^{L_j} , přičemž $G^{L_j} \geq G^{L_i}$ — tedy G^{L_j} je pro levého hráče alespoň tak dobrá pozice jako G^{L_i} . Tah do G^{L_i} levý hráč nikdy nezvolí. Jinými slovy, levý hráč vždy hledá maximální hodnotu a pravý vždy minimální hodnotu. Oba hráči jsou racionální, takže porovnáním odstraní všechny své nejhorší pozice nebo případně pozice se stejnou hodnotou, protože tím nezmění výsledek hry.

Poznámka. Po odstranění všech dominovaných tahů obecně nezůstane pouze jeden nebo žádný prvek v množině, protože některé hodnoty jsou vzájemně neporovnatelné (confused).

Definice 4.6 (Dominovaný tah převzatý z [10, str. 64–65]). *Levý tah G^{L_i} je dominován tahem G^{L_j} , pokud $G^{L_j} \geq G^{L_i}$. Pravý tah G^{R_i} je dominován tahem G^{R_j} , pokud $G^{R_j} \leq G^{R_i}$.*



Obrázek 4.4: Pozice modrého na řadě, obsahující dominovaný tah s indexem 0 dominovaným tahem s indexem 1

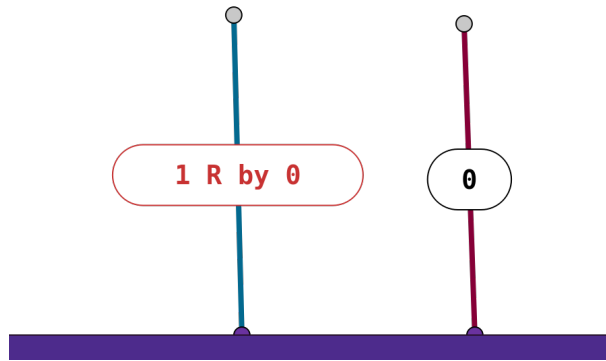
Na obrázku 4.4 existuje dominovaný tah levého (resp. modrého) hráče, přesněji tah do hrany 0 je dominován tahem do hrany 1. Pokud hraje levý (resp. modrý), jsou potom jen dva scénáře. Levý zničí hranu s indexem 0, čímž získá pozici s hodnotou 0. Pokud zničí hranu s indexem 1, získá pozici s hodnotou 1. Levý chce hodnotu pozice maximalizovat, a protože je racionální hráč, vybere si vždy tah do hrany s indexem 1.

Reverzibilní tahy

Druhý typ nadbytečného tahu je subtilnější. Levý hráč zahraje tah G^{L_i} , ale pravý hráč má odpověď $G^{L_i R_j}$ takovou, že $G^{L_i R_j} \leq G$ — výsledná pozice je pro levého hráče horší nebo stejná jako původní G .

Definice 4.7 (Reverzibilní tah převzatý z [10, str. 64–65]). *Levý tah G^{L_i} je reverzibilní přes $G^{L_i R_j}$, pokud existuje pravý tah $G^{L_i R_j}$ z pozice G^{L_i} takový, že $G^{L_i R_j} \leq G$. Symetricky, pravý tah G^{R_i} je reverzibilní přes $G^{R_i L_j}$, pokud $G^{R_i L_j} \geq G$.*

Reverzibilní tah nelze jednoduše odstranit — musí se *obejít* (bypass). Tah G^{L_i} se nahradí přímo všemi tahy $G^{L_1 R_1 L}$, které by levý hráč zahrál poté, co pravý anuluje jeho tah.



Obrázek 4.5: Pozice modrého na řadě, obsahující reverzibilní tah s indexem 1, reverzibilní přes tah s indexem 0

Vezmu příklad 4.5, kde jsou dvě hrany, jedna modrá a jedna červená, obě vycházející ze země. Jde se jednoduše přesvědčit o tom, že číslo takové pozice je 0. Na obrázku je naznačeno, že tah do modré hrany je reverzibilní. Proč? Zjednodušeně řečeno, reverzibilní tah je takový, na který existuje nějaká odpověď, která přivede danou pozici na *stejnou* nebo *horší* hodnotu. Pokud tedy levý hráč (resp. modrý) zničí modrou hranu, pravý (resp. červený) musí zničit červenou hranu a vrátit tím pozici zpět na číslo 0. Platí $0 \geq 0$, takže zničení modré hrany je reverzibilní zničením červené hrany. Symbolicky by to vypadalo následovně:

$$\begin{aligned} G = 0 &= \{-1 \mid 1\} = \{\{ \mid 0\} \mid \{0 \mid \}\}, \\ G^{L_1} &= \{ \mid 0\} \\ G^{L_1 R_1} &= 0 \\ G^{L_1 R_1} &\geq G \end{aligned}$$

Když je toto ověřeno, je nyní možnost nahradit tah G^{L_i} tahy $G^{L_1 R_1 L}$, v tomto případě tah levého hráče v nulové pozici, tedy žádný. Tím se získá:

$$G = \{ \mid \{0 \mid \}\}$$

Lze si povšimnout, že obrázek 4.5 je trochu zavádějící, protože stejným způsobem se lze přesvědčit o tom, že i tah pravého je reverzibilní. Toto je způsobeno vytvořenou aplikací, která ukazuje dominované a reverzibilní tahy pouze hráče na tahu. Ale stejným způsobem by se dalo přesvědčit, že po odstranění všech reverzibilních tahů by se získala rovnost

$$\{\{ \mid 0\} \mid \{0 \mid \}\} = \{ \mid \}.$$

Stojí za to zmínit, že odstraněním dominovaných tahů a obcházením reverzibilních tahů *se nemění* hodnota hry [5, věta 68, str. 110–111].

Existence a jednoznačnost kanonické formy

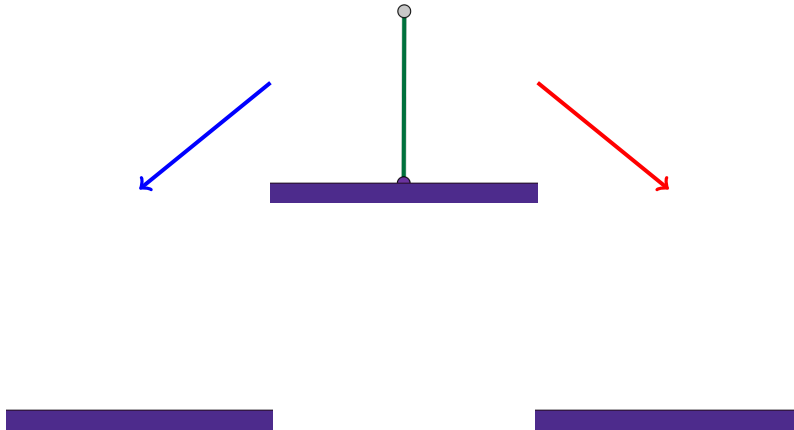
Definice 4.8 (Kanonická forma převzata z [10, str. 66]). *Krátká hra K je v kanonické formě, pokud žádná její podpozice neobsahuje dominované ani reverzibilní tahy.*

Pro každou krátkou hru G existuje právě jedna kanonická forma $K = G$. Tato forma se získá opakovaným odstraňováním dominovaných tahů a nahrazováním reverzibilních tahů jednoduššími ekvivalentními možnostmi, dokud již žádné takové úpravy nejsou možné [5, věta 69, str. 112].

4.2 Infinitesimální hodnoty

Dyadická čísla lze chápat jako speciální případ hodnot krátkých her. Některé herní pozice však nelze vyjádřit dyadickým číslem. Jejich hodnota musí být popsána obecnější krátkou hrou. Mezi takové hodnoty patří například *infinitesimální* hodnoty.

Všechny pozice v Hackenbushu lze vyjádřit jako dyadické číslo [4], které lze triviálně porovnávat, sčítat a provádět další jednoduché operace. Celý aparát krátkých her musí být použit na řešení Hotpotche právě proto, že obsahuje infinitesimální hodnoty a jejich součty, které musí být porovnány podle definice — na rozdíl od dyadických čísel.



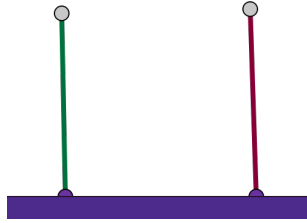
Obrázek 4.6: Ukázka rozkladu hry s jednou zelenou hranou vycházející ze země. Modrá šipka značí tah levého hráče do nulové hry vlevo dole, červená šipka analogicky značí tah pravého hráče do nulové hry vpravo dole.

Příkladem hry s infinitesimální hodnotou je obrázek 4.6, který zobrazuje hru s jednou zelenou hranou vycházející z kořenového vrcholu (země). Zelenou hranu mohou zničit oba hráči: pokud začne levý hráč, zničí zelenou hranu a dostane protihráče do pozice, ze které nelze provést žádný tah, a podle normální konvence prohrává. Pokud začne pravý hráč, analogicky zničí zelenou hranu a vyhraje. Tím jsou prošlé všechny možné pozice a lze říct, že ten, kdo hraje první, vyhrává. První instinkt je podívat se na nulovou pozici — tu ale prohrává hráč, který hraje první (nulová pozice je taková, kde nelze táhnout, tedy hráč na řadě prohrává). Hodnota na obrázku 4.6 tedy není nula ani žádné dyadické číslo.

Přepisem na formu krátké hry lze takové hře přiřadit hodnotu $\{0 \mid 0\}$, která si zaslouží speciální symbol $*$.

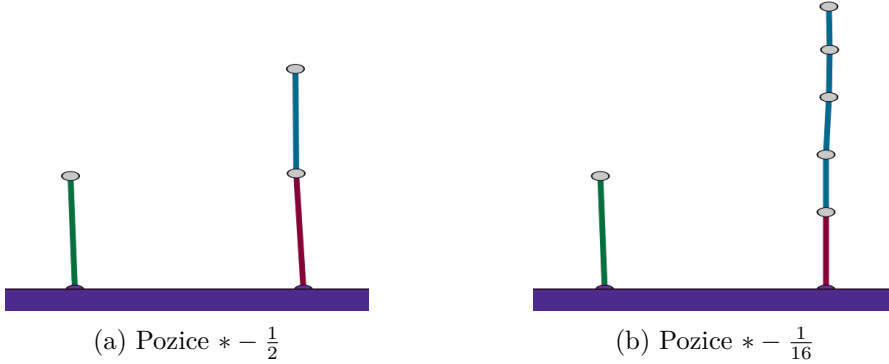
Další otázkou je, jak se taková hra porovnává s ostatními hodnotami. Standardně se vztahy porovnávají tak, že se ke hře přidá další pozice a sleduje se, jak se celková hodnota změní — nejlépe tak, aby součet byl roven nule. Tímto způsobem lze najít hodnotu hledaného výrazu nebo alespoň jeho vztah k ostatním hodnotám.

Mějme pozici znázorněnou na obrázku 4.7.



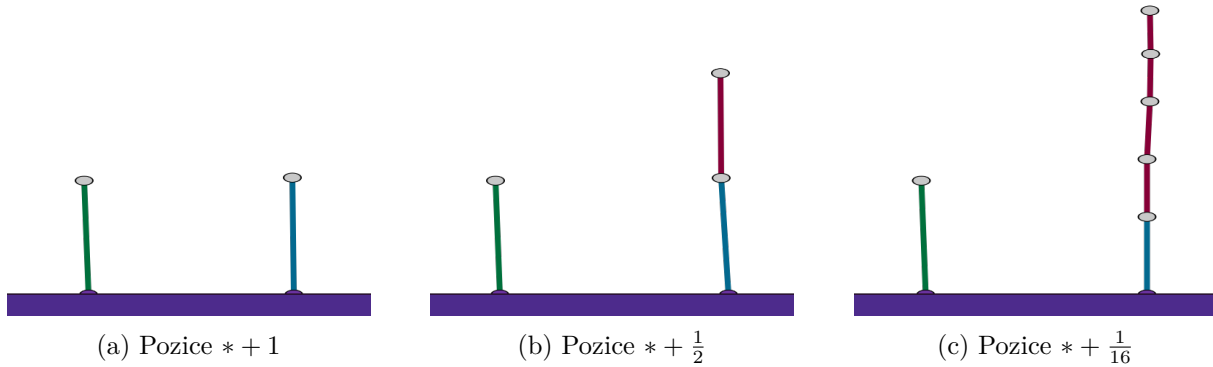
Obrázek 4.7: Pozice s hodnotou $* - 1$.

Taková hra má hodnotu $* + (-1)$. Lze se jednoduše přesvědčit, že tato pozice je výhrou pro červeného, tedy $* - 1 < 0$, odkud $* < 1$.



Obrázek 4.8: Ukázka porovnání $*$ s kladnými dyadickými čísly.

Stejně tak se lze přesvědčit na obrázku 4.8, že $* < \frac{1}{2}$, dále $* < \frac{1}{16}$, a obecně $* < \frac{1}{2^k}$ pro každé přirozené číslo k . Analogickým způsobem se lze přesvědčit, že $* > -1$, $* > -\frac{1}{2}$, $* > -\frac{1}{16}$, a obecně $* > -\frac{1}{2^k}$ pro každé přirozené číslo k , znázorněno na obrázcích 4.9.



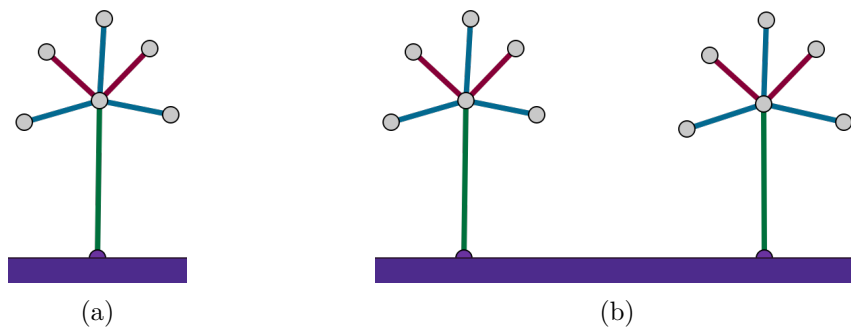
Obrázek 4.9: Ukázka porovnání $*$ se zápornými dyadickými čísly.

Dohromady tedy platí

$$-\frac{1}{2^k} < * < \frac{1}{2^k} \quad \text{pro každé } k \geq 0.$$

Jediné dosud zavedené číslo splňující tuto podmínku je 0, ale jak bylo ukázáno, $* \neq 0$. Vztah, který má $*$ k nule, se nazývá *neporovnatelnost* [1, str. 29–30] (angl. *confused with*): píšeme $* \parallel 0$. Hry, které jsou neporovnatelné s nulou ($G \parallel 0$), nazýváme *fuzzy*. Takové hry mají tu vlastnost, že vyhrává hráč, který začíná první [1, str. 28].

Hodnota $*$ se může sčítat, odčítat a porovnávat stejně jako každá jiná herní hodnota. Součet hodnot G a $*$ zapisujeme jako $G + *$ nebo zkráceně $G*$ (pozor, nezaměňovat s $*G$) [1, str. 39].



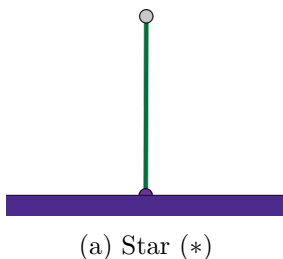
Obrázek 4.10: Pozice **a** má fuzzy hodnotu — vyhraje vždy hráč na řadě. Ale součet se stejnou pozicí **b** je výhra pro levého.

Na první pohled by se mohlo zdát, že pozice s fuzzy hodnotou znázorněna na obrázku 4.10a není ve prospěch žádného hráče a mělo by být nejlepší přiřadit jí hodnotu 0. Taková konvence by ale byla zavádějící, protože fuzzy pozice jsou často ve prospěch jednoho hráče, i když oba mohou vyhrát, pokud začnou první. Jak znázorňuje obrázek 4.10b, součet dvou fuzzy hodnot může být jednoznačně výhodný pro jednoho z hráčů.

Star. Hra $*$ = $\{0 \mid 0\}$ je fuzzy ($*$ $\parallel$ 0) a je svou vlastní negací: $* + * = 0$ [1, str. 66]. V kontextu Hackenbush odpovídá $*$ jedné zelené hraně nad zemí.

Definice 4.9 (Nimbery). Nimbery jsou definovány rekurzivně:

$$*0 = 0, \quad *1 = *, \quad *n = \{ *0, *1, \dots, *(n-1) \mid *0, *1, \dots, *(n-1) \}.$$



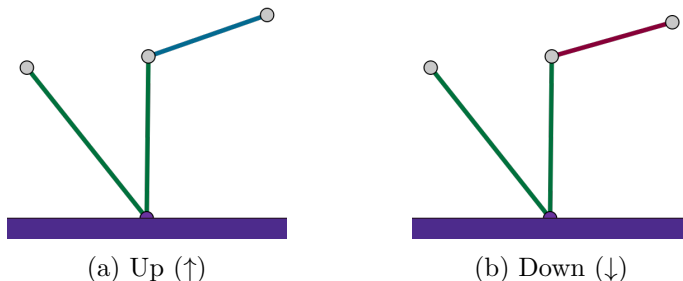
Obrázek 4.11: Pozice s hodnotou $*$ znázorněna ve hře Hotpotch.

Nimbery jsou hodnotami pozic Green Hackenbush a obecně imparciálních her. Řetěz k zelených hran nad zemí má hodnotu $*k$.

Up a Down. Conway zavedl další infinitezimální hodnoty, které nimbery nejsou. Motivací bylo popsat pozice, kde jeden hráč má jednoznačnou výhodu, ale ta je tak malá, že přidáním jakkoliv malého dyadického čísla se tato výhoda ztratí — situace typická pro Hothpotch.

Definice 4.10 (Up a Down).

$$\uparrow = \{0 \mid *\}, \quad \downarrow = \{* \mid 0\} = -\uparrow.$$



Obrázek 4.12: Pozice s hodnotami $\uparrow$ (a) a $\downarrow$ (b) znázorněny ve hře Hotpotch.

Proč $\uparrow > 0$? Levý hráč může táhnout do 0 (prohra pro hráče na tahu, tedy výhra pro druhého), zatímco pravý může táhnout pouze do $*$ (fuzzy — výhra pro prvního hráče). Pravý tedy nemá dobrou odpověď, a $\uparrow$ je kladné. Přitom $\uparrow < \frac{1}{n}$ pro každé $n > 0$. Tedy hodnota $\uparrow$ je kladná, ale menší než jakékoliv kladné dyadické číslo. Jde tedy o infinitezimál stejného charakteru jako $*$, ale tentokrát reálný (nenulový) ve prospěch levého hráče.

Platí $\uparrow \parallel *$ (jsou neporovnatelné), ale $\uparrow + \uparrow > *$.

Definice 4.11 (Infinitezimální hra, podle [10]). *Krátká hra G je infinitezimální, pokud $x > G > -x$ pro každé dyadické číslo $x > 0$.*

Vypracovaná knihovna dokáže základně rozpoznávat tyto speciální hodnoty a správně jim přiřadit speciální symbol. Algoritmus na převod krátké hry do znakové podoby však není formálně vypracovaný, nedokáže tedy obecně převést krátkou hru do nejjednodušší znakové podoby. Grafické rozhraní zároveň obsahuje i kalkulačku krátkých her, která používá další funkce z vypracované knihovny, jenž dokáže základně převést znakovou podobu krátké hry na vnitřní datovou strukturu krátké hry.

Algoritmus pro určení, zda je krátká hra infinitezimální, není z definice zřejmý. Knihovna tedy obsahuje i funkci, která používá metodu popsanou v [3, věta 4, str. 413], jenž dokáže určit, zda je daná krátká hra infinitezimální. Tuto funkci je možné využít v kalkulačce grafického rozhraní Hotpotche.

Kapitola 5

Dyadická čísla

John Conway nebyl prvním člověkem, který použil termín *surreálná čísla*. Ten se již vyskytuje v knize *Surreal Numbers* [7], kterou napsal Donald E. Knuth v roce 1974.

V té době Conway společně s Elwynem Berlekampem a Richardem Guyem pracoval na knize *Winning Ways*. Spolupráce ale nebyla jednoduchá, Berlekampův formální styl se střetával s Conwayovým hravějším přístupem a vznikaly spory o strukturu knihy. Berlekamp navíc cítil, že důraz na transfinite čísla výrazně zúží okruh čtenářů. V roce 1975 se proto Conway rozhodl napsat knihu *On Numbers and Games* sám, za jediný týden [10, str. 487].

Lze si všimnout, že se v této kapitole nemluví o *surreálných číslech*. Důvod je takový, že *surreálná čísla* jsou až moc obecnou třídou [10, str. 401] pro řešení problémů v této práci.

Co jsou surreálné čísla

Aby se dalo pochopit proč, musí se nejprve pochopit, co jsou *surreálná čísla*. **Dlouhá hra** $x = \left\{ x^L \mid x^R \right\}$ je *surreálné číslo*, pokud $\forall l \in x^L, \forall r \in x^R : l < r$. [10, str. 401].

Nehodí se o takovém vysvětlení surreálných čísel říkat, že se jedná o definici, protože formálně není jasné, co jsou dlouhé hry.

Dlouhé hry mohou mít obecně *nekonečně* mnoho možných pozic [10, str. 398]. Takové hodnoty tato práce nepoužívá a ani nemůže, protože některé praktické vlastnosti krátkých her obecně neplatí pro dlouhé hry, tedy ani pro surreálná čísla.

Důvod, proč tato kapitola obsahuje krátkou sekci o surreálných číslech, je ten, aby čtenář nebyl zmaten terminologií při čtení dalších zdrojů.

Mimo krátké hry tato práce používá druhý typ hodnot, které se nazývají *dyadická čísla*.

5.1 Definice dyadického čísla

Definice 5.1 (Dyadické číslo). *Racionální číslo je dyadické, pokud jeho jmenovatel je (v základním tvaru) mocnina dvou, včetně jmenovatele s hodnotou $2^0 = 1$. Množina dyadických čísel je*

$$\mathbb{D} = \left\{ \frac{m}{2^n} : m \in \mathbb{Z}, n \in \mathbb{N}_0 \right\}.$$

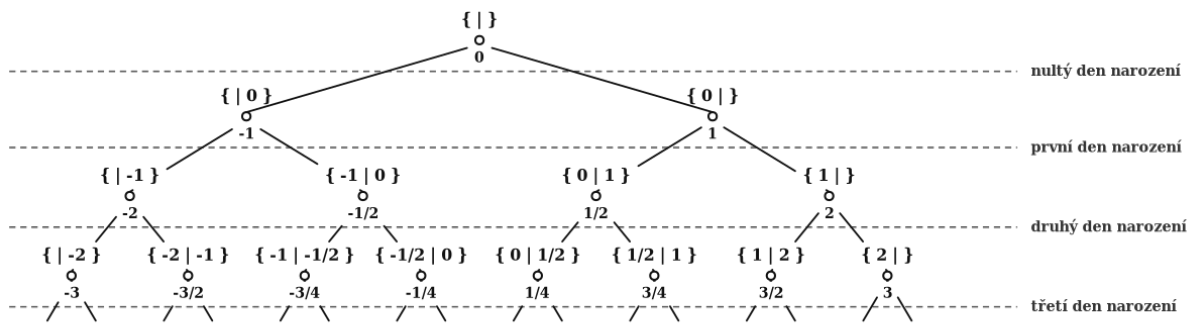
Krátká hra $G = \left\{ G^L \mid G^R \right\}$ je **číslo**, pokud $\forall l \in G^L, \forall r \in G^R : l < r$ [10, def. 3.1, str. 69]. Taková hra se chová jako číslo v tom smyslu, že zjednodušeně platí „ $G^L < G < G^R$ “ [10, tvrzení 3.2, str. 69]. Tedy hodnota hry leží striktně mezi jejími levými a pravými možnostmi.

Jak uvádí Siegel [10, str. 70], množina všech takových krátkých her tvoří podgrupu grupy $(\mathbb{G}, +)$, která je izomorfní právě s $\mathbb{D}$. Z tohoto důvodu lze ztotožňovat tuto podgrupu s odpovídajícím dyadickým číslem a psát přímo $\frac{m}{2^n}$ místo herní hodnoty. Tato podgrupa je jednoduše označena symbolem $\mathbb{D}$.

Nejdůležitějším rozdílem čísel oproti krátkým hrám je, že čísla (resp. dyadická čísla [10, str. 70]) jsou lineárně uspořádaná [5, str. 15]

Conway ve své knize *On Numbers and Games*, kde představil formálnější popis her, používal pojem „číslo“ namísto dnes známých surreálných čísel. Název *surreálná čísla* zavedl D. E. Knuth tři roky předtím, ale tyto pojmy byly v té době čerstvé, a Conway tedy tento výraz ve své první knize nepoužil.

Den narození - Birthday



Obrázek 5.1: Stromová reprezentace dyadických čísel, kde vodorovné čárkované čáry vyznačují jednotlivé dny narození

Definice 5.2 (Den narození). Den narození hry G , značený $b(G)$, je minimální počet rekurzivních kroků potřebných k sestrojení G od nulové hry 0. Říkáme, že G se narodila v den n , pokud $b(G) = n$.

Příklad 5.1 (Den narození základních her).

$$\begin{array}{ll}
 b(0) = 0 & \text{základ rekurze} \\
 b(1) = b(-1) = 1 & \text{sestrojitelné z } \{0\} \\
 b(\frac{1}{2}) = 2 & \text{vyžaduje 1 z dne 1}
 \end{array}$$

Den narození odpovídá pořadí, ve kterém jsou dyadická čísla konstruována. Každé číslo vzniká z již existujících hodnot a jeho den narození je o jednu větší než maximum dnu narození jeho možností:

$$b(x) = \max(1 + b(L), 1 + b(R)).$$

Pravidlo jednoduchosti

Pro číselnou krátkou hru

$$G = \left\{ G^L \mid G^R \right\}$$

platí, že její hodnota je nejjednodušší číslo, které je větší než všechny levé možnosti a zároveň menší než všechny pravé možnosti [5, str. 22]. Tedy hledáme nejjednodušší číslo x splňující

$$\forall l \in G^L, \forall r \in G^R : \quad l < x < r.$$

Pod pojmem nejjednodušší se zde myslí číslo s nejmenším dnem narození.

Příklad 5.2.

$$\begin{aligned} \{0 \mid 1\} &= \frac{1}{2}, & \{1 \mid 2\} &= \frac{3}{2}, & \left\{ 2 \mid 2\frac{1}{2} \right\} &= 2\frac{1}{4}, & \left\{ -3 \mid -3\frac{1}{2} \right\} &= -3\frac{1}{4}, \\ \{2 \mid 4\} &= 3, & \{-1 \mid 1\} &= 0, & \{-2 \mid 0\} &= -1, & \{-1 \mid 2\} &= 0, \\ \{2 \mid \} &= 3, & \{-100 \mid \} &= 0, & \left\{ \mid \frac{5}{2} \right\} &= 2, & \{ \mid -2 \} &= -3. \end{aligned}$$

Vezmu příklad $\{-1 \mid 2\} = 0$. Mezi čísla -1 a 2 se nachází nekonečně mnoho možných čísel, například $-\frac{1}{2}$ nebo $1\frac{1}{4}$, ale všechna tato čísla mají pozdější den narození než 0 (viz obrázek 5.1), pro kterou také platí, že $-1 < 0 < 2$.

Tímto způsobem jsou sestrojena všechna čísla v příkladech 5.2.

Kapitola 6

Návrh a implementace pomocí dyadických čísel

Tato kapitola se týká pouze číselného případu odpovídajícího Hackenbushi. Jak bylo uvedeno v předchozí kapitole, konečné pozice této varianty mají hodnoty v množině dyadických racionálních čísel, a proto není nutné je reprezentovat jako obecné krátké hry.

Cílem této části není dostat se k obecnému řešiči (solveru) pro Červeno-Modro-Zelenou variantu Hackenbushe (též Hotpotch), ale ukázat jinou možnou metodu řešení tohoto problému.

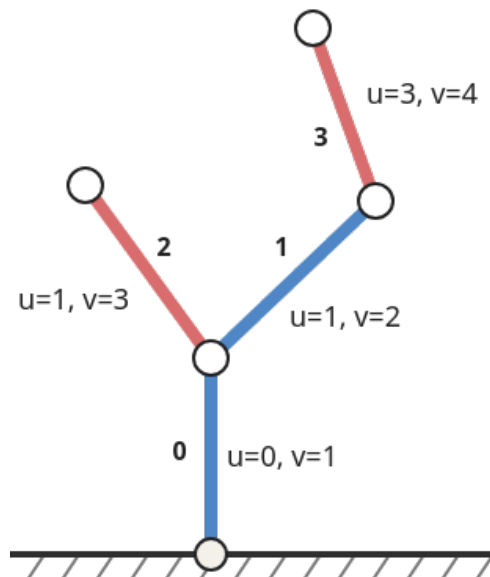
6.1 Datové reprezentace grafu Hackenbushe

Jediný dynamický proces při procházení tahů je mazání hran. Tato vlastnost umožňuje ponechat popis struktury grafu zcela statický. Informace o tom, zda je hrana živá (tj. dosud nesmazaná), je zakódována jako aktivní bit v n -bitovém celém čísle, dále označovaném typem `edge_mask_t`. Bit na pozici k odpovídá hraně s indexem k ; odebrání hrany je tedy jediná bitová operace.

Zde zavádím pojem *surová hra*. Surovou hrou budeme rozumět jakoukoli datovou reprezentaci hry, *nevyjádřenou* datovou reprezentací krátké hry. Datová reprezentace krátké hry je popsána v sekci 7.1. Surová hra Hackenbushe je reprezentována strukturou `BaseGraph`, která uchovává pouze neměnné informace o propojení hran, počtu hran a počtu vrcholů:

```
1 // src/dyadics/dyadics/C/raw_game.h
2 // src/short-games/C/games/hotpotch/hotpotch.h
3 typedef struct {
4     uint8_t u;
5     uint8_t v;
6     EdgeColor color;
7 } Edge;
8
9 typedef struct {
10     uint8_t num_vertices;
11     uint8_t num_edges;
12     Edge edges[MAX_EDGES];
13 } BaseGraph;
```

Výpis 6.1: Datová reprezentace grafu Hackenbushe



Obrázek 6.1: Datová reprezentace Hackenbushe a její vizuální ukázka s 5 vrcholy a 4 hranami.

Tato metoda vyjadřuje hodnoty pozic Hackenbushe pouze dyadickými čísly.

$$\frac{m}{2^n}$$

kde $m \in \mathbb{Z}$, $n \in \mathbb{N}_0$. Tato reprezentace umožňuje porovnávání a konstrukci nových hodnot. Tento přístup je použit v souboru `src/dyadics/C/dyadic.h`, kde je vytvořena následující struktura:

```
1 // src/dyadics/C/dyadics.h
2 // G = num / 2^exp
3 typedef struct {
4     long long num;
5     int exp;
6 } Dyadic;
```

Výpis 6.2: Struktura dyadického čísla.

6.1.1 Pravidlo jednoduchosti - nejjednodušší dyadické číslo mezi dyadickými čísly

Řekněme, že možné pozice levého a pravého jsou již induktivně vyjádřeny jako dyadické číslo D_L a D_R , kde $D_L, D_R \in \mathbb{D}$. Ekvivalentní zápis této formulace je pomocí krátké hry vyjádřen jako $\{D_L \mid D_R\}$. Zápis pomocí krátkých her ale není potřeba, protože hodnotu $\{D_L \mid D_R\}$ lze vyjádřit jako nejjednodušší dyadické číslo $D \in \mathbb{D}$, tak aby „ $D_L < D < D_R$ “. Podmínkou převodu v rámci implementace je $|D_L|, |D_R| \in \{0, 1\}$ a na základě počtu prvků jsou potom rozděleny algoritmy 1, 2 a 3.

Této podmínky lze dosáhnout právě proto, že dyadická čísla jsou lineárně uspořádaná, jak je uvedeno v sekci 5.1. Tím je zajištěno, že množina dyadických čísel vždy obsahuje nějaký minimální a maximální prvek. Při procházení prvků levých možností lze tedy držet pouze jeden prvek L_{max} , a analogicky při procházení pravých možností pouze R_{min} , a použít je pro výpočet dyadického čísla této pozice. Tímto způsobem se lze vyhnout stromové struktuře krátkých her a celý proces zjednodušit.

Popis nalezení nejjednoduššího dyadického čísla

Nalezení nejjednoduššího dyadického čísla mezi dvěma dyadickými čísly není triviální, proto je zde uveden postup, jak se k takovému algoritmu dá dojít.

Nechť $a < b$ jsou dyadická čísla. Hledá se nejjednodušší dyadické číslo x , takové že

$$a < x < b.$$

Pod pojmem *nejjednodušší* se zde myslí dyadické číslo s nejmenším možným dnem narození. Pro dyadická racionální čísla je to ekvivalentní tomu, že po zkrácení má tvar

$$x = \frac{n}{2^k}$$

s nejmenším možným exponentem k . Pro dané k chceme zjistit, zda některý násobek $\frac{1}{2^k}$ leží přísně mezi a a b . To znamená, že hledáme celé číslo n splňující

$$a < \frac{n}{2^k} < b.$$

Jedna z možností, jak dojít k obecnému algoritmu, je postupně zkoušet porovnávat nejbližší možné a a b s přesností $\frac{1}{2^k}$. Takové zaokrouhlení čísla x na nejbližší nižší násobek $\frac{1}{2^k}$ lze vyjádřit jako

$$\frac{\lfloor x \cdot 2^k \rfloor}{2^k}.$$

Nejmenší násobek $\frac{1}{2^k}$ striktně větší než a je $n_{\min}$ a největší násobek $\frac{1}{2^k}$ striktně menší než b je $n_{\max}$, kde

$$n_{\min} = \frac{\lfloor a \cdot 2^k \rfloor + 1}{2^k}, \quad n_{\max} = \frac{\lceil b \cdot 2^k \rceil - 1}{2^k}.$$

Poznámka. Funkce $\lfloor \cdot \rfloor$ zde značí vždy zaokrouhlení směrem k $-\infty$, tedy dolní celou část čísla. Naopak funkce $\lceil \cdot \rceil$ značí vždy zaokrouhlení směrem k $+\infty$, tedy horní celou část čísla.

Hodnoty $n_{\min}$ a $n_{\max}$ označují meze hledaného intervalu. Podmínka existence kandidáta je

$$n_{\min} \leq n_{\max}.$$

Pokud je tato podmínka splněna, potom nejjednodušší dyadické číslo striktně větší než a a striktně menší než b je $n_{\min}$.

Tento algoritmus lze následně zjednodušit odložením dělení jmenovatelem, protože

$$\begin{aligned} n_{\min} &\leq n_{\max} \\ \frac{\lfloor a \cdot 2^k \rfloor + 1}{2^k} &\leq \frac{\lceil b \cdot 2^k \rceil - 1}{2^k} \\ \lfloor a \cdot 2^k \rfloor + 1 &\leq \lceil b \cdot 2^k \rceil - 1. \end{aligned}$$

Protože hodnoty $k = 0, 1, 2, \dots$ procházíme vzestupně, první nalezené řešení má nejmenší možný jmenovatel a je tedy nejjednodušší. Celý postup je shrnut v algoritmu 1.

Algorithm 1 Nejjednodušší dyadické číslo mezi dvěma hodnotami

Require: Dyadická čísla a, b , kde $a < b$

Ensure: Dyadické číslo x splňující $a < x < b$

```

1: for  $k = 0, 1, 2, \dots$  do
2:   Urči interval celých čísel
      $n_{\min} = \lfloor a \cdot 2^k \rfloor + 1, \quad n_{\max} = \lceil b \cdot 2^k \rceil - 1$ 
3:   if  $n_{\min} \leq n_{\max}$  then
4:     return  $x = \frac{n_{\min}}{2^k}$ 
5:   end if
6: end for
```

Nejjednodušší dyadické číslo nad hodnotou

V případě, že existuje pouze levá možnost, hledá se nejjednodušší dyadické číslo větší než daná hodnota.

Algorithm 2 Nejjednodušší dyadické číslo nad hodnotou

Require: Dyadické číslo a

Ensure: Dyadické číslo x splňující $x > a$

```

1: Urči celé číslo  $q = \lceil a \rceil$ 
2: if  $q > a$  then
3:   return  $q$ 
4: else
5:   return  $q + 1$ 
6: end if
```

Nejjednodušší dyadické číslo pod hodnotou

Analogicky, pokud existují pouze pravé možnosti, hledá se nejjednodušší dyadické číslo menší než daná hodnota.

Algorithm 3 Nejjednodušší dyadické číslo pod hodnotou

Require: Dyadické číslo b

Ensure: Dyadické číslo x splňující $x < b$

```
1: Urči celé číslo  $q = \lfloor b \rfloor$ 
2: if  $q < b$  then
3:   return  $q$ 
4: else
5:   return  $q - 1$ 
6: end if
```

Tyto tři operace odpovídají definici hodnoty dyadického čísla a jsou použity při výpočtu hodnot pozic v rámci řešiče.

Převod surové hry na dyadické číslo

Hackenbush má tu vlastnost, že hodnota každé jeho pozice je dyadické číslo. Stačí tedy převést surovou hru na dyadické číslo a porovnat výsledek s nulou

Pro převedení surové hry na dyadické číslo poslouží následující rekurzivní algoritmus.

Algorithm 4 Rekurzivní výpočet hodnoty pozice Hackenbush

Require: Graf g , maska živých hran $live_mask$

Ensure: Dyadická hodnota pozice

```
1: if  $live\_mask \in memo$  then
2:   return  $memo[live\_mask]$ 
3: end if
4: if  $live\_mask = 0$  then
5:   return 0
6: end if
7:  $has\_left \leftarrow false, has\_right \leftarrow false$ 
8: for každou hranu  $e \in live\_mask$  do
9:    $child \leftarrow cleanup(g, live\_mask \setminus \{e\})$ 
10:   $v \leftarrow solve(g, child)$ 
11:  if hrana  $e$  je modrá then
12:    if  $\neg has\_left$  or  $v > L_{max}$  then  $L_{max} \leftarrow v; has\_left \leftarrow true$ 
13:    end if
14:  else
15:    if  $\neg has\_right$  or  $v < R_{min}$  then  $R_{min} \leftarrow v; has\_right \leftarrow true$ 
16:    end if
17:  end if
18: end for
19: if  $\neg has\_left \wedge \neg has\_right$  then  $result \leftarrow 0$ 
20: elseif  $has\_left \wedge \neg has\_right$  then  $result \leftarrow simplestAbove(L_{max})$ 
21: elseif  $\neg has\_left \wedge has\_right$  then  $result \leftarrow simplestBelow(R_{min})$ 
22: elseif  $L_{max} < R_{min}$  then  $result \leftarrow simplestBetween(L_{max}, R_{min})$ 
23:  $memo[live\_mask] \leftarrow result$ 
24: return  $result$ 
```

Funkce nejprve rekurzivně vyřeší všechny pozice dosažitelné jedním tahem a z jejich hodnot sestaví hodnotu aktuální pozice. Hloubka rekurze je shora omezena počtem hran, protože každý tah odstraní právě jednu hranu a hra je konečná.

Výpočet začíná kontrolou hashovací tabulky *memo*. Pokud pozice není nalezena, zkontroluje se, zda je živá maska nulová. To odpovídá nulové hře a zároveň slouží jako hlavní ukončovací podmínka rekurze.

Poté se iteruje přes všechny živé hrany (řádek 8). Pro každou hranu se vytvoří nová pozice jejím odstraněním a rekurzivně se vypočítá její hodnota. Zároveň se v tomto cyklu průběžně udržují agregátory $L_{\max}$ a $R_{\min}$ (řádky 11 a 14), čímž se implicitně odstraňují dominované tahy.

Kapitola 7

Návrh a implementace pomocí krátkých her

Předchozí kapitoly zavedly dva pohledy na hodnotu pozice. V číselném případě Hackenbushe stačí pracovat s dyadickými racionálními čísly, obecný Hotpotch však vyžaduje plnou reprezentaci krátkých her. Tato kapitola popisuje, jak tento obecný aparát převést do implementace řešiče.

7.1 Od surové hry ke krátké hře

Aby bylo možné určit výsledek hry, je třeba surovou hru převést do tvaru krátké hry. Z definice krátké hry plyne rekurzivní struktura: hra je popsána množinou jednodušších her, které jsou opět popsány množinou ještě jednodušších her, až se dospěje do hry, ve které nelze zahrát žádný tah — v Hotpotchi se jedná o pozice s bitmaskou 0.

Přirozenou reprezentací je tedy uložit pro každou pozici ukazatele na její levé a pravé možnosti. Každá pozice obsahuje pouze informace o svých jednodušších hrách. Nikde, krom pozic které už žádné další hry neobsahují, ani nedává smysl ukládat informaci o pozici pomocí bitmasky. V listech lze uložit informaci o tom, že se jedná o pozici, ze které není možné udělat další tah, tuto informaci lze ale získat i z počtu L_count a R_count , kde pokud $L_count = R_count = 0$, tak se jedná o takovou nulovou hru a dodatečná informace o tomto je zbytečná. Lze si tedy všimnout, že hra reprezentována krátkou hrou *neobsahuje* informaci o vnitřní struktuře dané hry.

Krátká hra je v implementaci reprezentována strukturou `Game`:

```
1 // src/short-games/C/short_game.h
2 typedef struct Game {
3     int L_count;
4     int R_count;
5     struct Game **left; /* Pole ukazatelů na levé možnosti. */
6     struct Game **right; /* Pole ukazatelů na pravé možnosti. */
7 } Game;
```

Výpis 7.1: Struktura uzlu krátké hry.

Surová reprezentace Hotpotche je nadále stejná, jako reprezentace Hackenbushe ve výpisu 6.1. Pouze byla přidána nová možnost barvy hrany.

7.2 Výsledek hry

Výsledek hry G určuje, který hráč vyhraje při optimální hře obou stran. Existují čtyři třídy výsledků v závislosti na vztahu G k nule. Důležité je, že všechny čtyři třídy lze vyjádřit výhradně pomocí porovnání $\geq$, která je k dispozici v řešiči.

Definice 7.1 (Výsledek hry). *Nechť G je krátká hra a $0 = \{ \mid \}$ nulová hra. Potom platí:*

$$\begin{aligned} G > 0 &\iff G \in \mathcal{L} \iff G \geq 0 \wedge \neg(0 \geq G), \\ G < 0 &\iff G \in \mathcal{R} \iff 0 \geq G \wedge \neg(G \geq 0), \\ G = 0 &\iff G \in \mathcal{P} \iff G \geq 0 \wedge 0 \geq G, \\ G \parallel 0 &\iff G \in \mathcal{N} \iff \neg(G \geq 0) \wedge \neg(0 \geq G). \end{aligned}$$

Stačí tedy provést dvě porovnání: $G \geq 0$ a $0 \geq G$. Jejich kombinace jednoznačně určí třídu výsledku:

Tabulka 7.1: Určení třídy výsledku ze dvou volání $\geq$.

$G \geq 0$	$0 \geq G$	Třída	Výsledek
1	0	$\mathcal{L}$	vyhraje levý bez ohledu na pořadí
0	1	$\mathcal{R}$	vyhraje pravý bez ohledu na pořadí
1	1	$\mathcal{P}$	vyhraje druhý hráč
0	0	$\mathcal{N}$	vyhraje první hráč

V implementaci solveru se výsledek hry určuje přesně tímto způsobem, tedy dvěma voláními `game_geq` s hodnotou 0 jako referenčním bodem, bez jakéhokoli explicitního výpočtu výherní strategie.

7.3 Explicitní rekurzivní převod surové hry

V minulé sekci 7.2 byla popsána myšlenka o tom, jak vyřešit surovou hru, a to převést tuto surovou hru do matematické podoby krátké hry a dvakrát porovnat s nulou. Jak tedy převést surovou hru na krátkou hru? Můj první pokus používal explicitní metodu, tím je myšleno, že algoritmus pro převod obsahuje prvky specifické pro Hotpotch, například funkci `cleanup`, která odstraní všechny hrany, které již nejsou propojené se zemí. Takový způsob není tolik zajímavý, ale díky němu lze lépe pochopit další způsob řešení.

Require: Graf g , maska živých hran $live_mask$ **Ensure:** Ukazatel na krátkou hru G

```
1: if  $(g, live\_mask) \in position\_cache$  then
2:   return  $position\_cache[g, live\_mask]$ 
3: end if
4: Inicializuj pole  $L \leftarrow [], R \leftarrow []$ 
5: for každou živou hranu  $e \in live\_mask$  do
6:    $child\_mask \leftarrow cleanup(g, live\_mask \setminus \{e\})$ 
7:    $child \leftarrow solve\_component(g, child\_mask)$ 
8:    $c \leftarrow g.edges[e].color$ 
9:   if  $c = \text{modrá}$  nebo  $c = \text{zelená}$  then
10:     $L.append(child)$ 
11:   end if
12:   if  $c = \text{červená}$  nebo  $c = \text{zelená}$  then
13:     $R.append(child)$ 
14:   end if
15: end for
16:  $G \leftarrow game\_make(L, R)$ 
17:  $G \leftarrow game\_canonicalize(G)$ 
18:  $position\_cache[g, live\_mask] \leftarrow G$ 
19: return  $G$ 
```

Funkce nejprve rekurzivně vyřeší všechny pozice dosažitelné jedním tahem a z jejich hodnot sestaví hodnotu aktuální pozice. Hloubka rekurze je shora omezena počtem hran, protože každý tah odstraní právě jednu hranu a hra je konečná.

Výpočet začíná kontrolou hashovací tabulky `position_cache`. Pokud pozice není nalezena, iteruje se přes všechny živé hrany. Pro každou hranu se vytvoří nová pozice jejím odstraněním, rekurzivně se vypočítá její hodnota a výsledek se zařadí do pole levých nebo pravých možností podle barvy hrany — zelená hrana se přidá do obou, proto jsou použity dvě nezávislé podmínky místo `if-else`. Po zpracování všech hran se z nasbíraných možností sestaví hra voláním `game_make` (řádek 16) a výsledek se kanonizuje. Lokální pole L a R (řádek 4) jsou pole ukazatelů typu `Game*` s životností omezenou na dobu trvání volání funkce. Funkce `game_make` proto ukazatele překopíruje do nově alokované struktury na haldě. Funkce `game_make` využívá techniky mělké kopie (shallow copy), mohla by vyvstát otázka, jak může takový postup vytvořit pozici na haldě, když následnické uzly nejsou zkopírovány. Důvodem je, že tato kopie je využita už u nulové pozice (pozice, kde $L_{count} = R_{count} = 0$), taková pozice je jediná, která je hluboce zkopírována (deep copy). Protože má ale tato hra konečný počet tahů, každá sekvence tahů vždy skončí u nulové pozice, tedy u hry, která je hluboce zkopírována.

Podobně jako v algoritmu pro Červeno-Modrý Hackenbush (algoritmus 4) by mohly být dominované pozice odstraněny už při připojení dalšího tahu do polí L a R . Narozdíl od Červeno-Modrého Hackenbushe tímto způsobem obecně nezůstanou pouze pozice s 0 nebo 1 prvky, protože některé hodnoty mohou být vzájemně neporovnatelné (viz sekci 4.2). Tímto způsobem by se musela přidávaná pozice porovnávat vůči více pozicím při každém přidávání do polí L a R . Taková komplexita navíc nepřináší žádnou výhodu.

Kanonizace se provádí ihned po sestavení každé podpozice, nikoli až na výsledné hře. Pokud by se kanonizace odkládala, ukládaly by se do `position_cache` nekanonizované mezivýsledky, což by tabulku zaplňovalo výrazně rychleji. Stavový prostor Hackenbush je natolik velký, že předčasné vyčerpání volných míst v tabulce by ji zbavilo užitečnosti dříve, než by výpočet dospěl k výsledku. Průběžná kanonizace tedy snižuje paměťové nároky algoritmu za cenu vyšší výpočetní režie.

7.4 Memoizace a hashovací tabulky

Memoizace je nejčastěji optimalizační technika pro zrychlení programů. Jedná se o proces, kdy si funkce automaticky ukládá výsledky předešlých výpočtů. Tradičně se jedná o hashovanou tabulku, ve které je postupně uložena dvojice vstupů a výsledků [8].

Solver udržuje pět hashovacích tabulek s lineárním řešením kolizí.

Tabulka `position_cache` ukládá již vyřešené pozice Hackenbushe, aby nebylo nutné stejnou pozici opakovaně přepočítávat. Tato tabulka odpovídá přiřazení

V kapitole 8 jsou všechny algoritmy zobecněny pro libovolné kombinatorické hry, společně s formou této tabulky. Tato tabulka odpovídá přiřazení

Tabulka `game_geq_cache` memoizuje výsledky porovnání her vzhledem k relaci $\geq$. Prakticky se jedná o největší mezipaměť, protože operace $\geq$ je při kanonizaci volána velmi často (viz sekce 7.7). Tato tabulka odpovídá přiřazení

Tabulka `game_add_cache` uchovává výsledky disjunktivního součtu her, protože stejné kombinace her se během výpočtu objevují opakovaně. Tato tabulka odpovídá přiřazení

Tabulka `game_canon_cache` ukládá výsledky kanonizace her. Kanonizace zahrnuje odstraňování dominovaných a reverzibilních tahů a je výpočetně náročná, proto je výhodné kanonizovat nejvýše jednou. Tato tabulka odpovídá přiřazení

`(BaseGraph, edge_mask_t)`

↓
`*Game`

`(void *, void *)`

↓
`*Game`

`(*Game, *Game)`

↓
`{0, 1}`

`(*Game, *Game)`

↓
`*Game`

`*Game`
↓
`*Game`

Další hashovací tabulky

Tabulka `game_intern_cache` je speciální hashovací tabulka, bez které **nelze** správně převést surovou hru na krátkou hru v této implementaci. Těžko ji zařadit mezi klasické memoizační techniky, protože řešení bez ní (nebo řešení s plnou tabulkou) je fatální. Jejím úkolem je zajistit, že každá hra má v paměti právě jednu unikátní reprezentaci. Právě proto je typ `Game` explicitně reprezentován jako `*Game`, tedy ukazatel na krátkou hru, což umožňuje některé operace realizovat pomocí adresové aritmetiky.

Tato tabulka odpovídá přiřazení

`*Game`
↓
`*Game`

kteřé každé strukturální reprezentaci hry přiřazuje její kanonickou instanci. Pokud již v tabulce existuje hra se stejnou strukturou, je vrácena existující instance. Jinak je nová hra do tabulky vložena.

Tento přístup odpovídá technice známé jako *hash-consing* a zajišťuje, že dvě strukturálně identické hry jsou reprezentovány stejným ukazatelem. Díky tomu lze rovnost her testovat v konstantním čase pomocí porovnání ukazatelů a zároveň dochází k výraznému snížení paměťových nároků.

Před vložením do `intern_cache` jsou levé a pravé tahy seřazeny podle hodnoty pointeru, což zaručuje stabilní klíč nezávislý na pořadí, ve kterém byly tahy přidány. Pokud tabulka již obsahuje strukturálně identický uzel, vrátí jeho ukazatel. Jinak vloží nový.

7.5 Kanonizace a její důsledky

V sekci 7.3, algoritmus 5, se nachází řádek 17 s funkcí `game_canonicalize()`, která očekává krátkou hru jako strukturu `Game` definovanou v podsekci 7.1.

Kanonizace probíhá odstraňováním dvou typů redundantních tahů. Prvním jsou *dominované tahy*: levá možnost L' je dominována, pokud existuje jiná levá možnost L'' , taková, že $L'' \geq L'$. Levý ji nikdy nemá důvod hrát, protože L'' mu dává alespoň stejně dobrou pozici. Symetricky pro pravé možnosti. Druhým typem jsou *reverzibilní tahy*: levá možnost L' je reverzibilní, pokud existuje pravá odpověď $L'^R \leq G$. Pravý může po takovém tahu levému okamžitě odpovědět a dostat se do pozice, která je pro levého nejvýše tak dobrá jako výchozí hra G . Takový tah levého lze nahradit přímo levými možnostmi L'_R , čímž se zkrátí herní strom bez změny hodnoty.

Kanonizace probíhá ve smyčce, dokud nenastane žádná změna. Průchody odstraňující dominované tahy se střídají s obcházením reverzibilních tahů. Po každé změně je nutné smyčku spustit znovu, protože obcházení reverzibilního tahu zavádí nové možnosti, které mohou být dominované nebo reverzibilní.

Nejjednodušší dyadické číslo krátkých her v kanonické formě

Pokud je výsledná hodnota kanonizace krátká hra $G = \{G^L \mid G^R\}$ číslo ($\forall l \in G^L, \forall r \in G^R : l < r$), potom důsledkem kanonizace je, že zbývající meze $G_{max}^L = \max G^L$ a $G_{min}^R = \min G^R$ jsou nejbližší možné a zároveň počet levých a pravých možností je buď 0 nebo 1 – čísla G_{max}^L a G_{min}^R (částečně odůvodněno v sekci 6.1.1)

To má přímý důsledek pro výpočet nejjednoduššího dyadického čísla krátkých her. Pokud mají možnosti levého a pravého pouze jeden prvek, jedná se jednoduše o jejich průměr.

$$G = \frac{G_{max}^L + G_{min}^R}{2}.$$

Knihovna obsahuje v souboru `src/short-games/C/singletons.c` funkci `get_dyadic_value`, která převede krátkou hru na klasické dyadické číslo. V sekci 5.1 bylo popsáno, že číselné krátké hry tvoří podgrupu grupy krátkých her $\mathbb{G}$ vzhledem k operaci součtu her. Tato podgrupa je izomorfní grupě dyadických racionálních čísel $\mathbb{D}$. Proto ji dále ztotožňuji přímo s množinou $\mathbb{D}$ a hodnoty krátkých her, jako jsou G_{max}^L a G_{min}^R , zapisuji rovnou jako odpovídající dyadická čísla.

Upozorňuji čtenáře na to, že používám terminologii z knihy [10], kde číslo je krátká hra, která splňuje podmínku popsanou v 5.1. Naopak dyadické číslo je to číslo, které všichni dobře známe.

Pokud má číslo kanonickou formu a oba hráči mají právě jednu možnost G_{max}^L a G_{min}^R , potom pro převod na dyadické číslo stačí zprůměrovat G_{max}^L a G_{min}^R .

Příklad 7.1 (Převod čísla v kanonické formě na číslo dyadické).

$$\begin{aligned} \{G^L \mid G^R\} &= \{3 \mid 4\} \\ G_{max}^L &= 3, \quad G_{min}^R = 4 \\ G &= \frac{3 + 4}{2} = 3.5 \end{aligned}$$

Algorithm 6 Kanonizace hry G

Require: Hra $G = \{G^L \mid G^R\}$, všechny podpozice již v kanonické formě

Ensure: Kanonická forma G

```
1: if  $G \in \text{canon\_cache}$  then return  $\text{canon\_cache}[G]$ 
2: end if
3: repeat
4:    $changed \leftarrow false$ 
5:   for každý  $G^{L_i} \in G^L$  do ▷ Reverzibilní levé tahy
6:     for každý  $G^{L_i R_j} \in G^{L_i R}$  do
7:       if  $G \geq G^{L_i R_j}$  then ▷  $G^{L_i R_j} \leq G$ 
8:         Nahraď  $G^{L_i}$  všemi  $G^{L_i R_j L_k}$ 
9:          $changed \leftarrow true$ ; break
10:      end if
11:    end for
12:  end for
13:  for každý  $G^{R_i} \in G^R$  do ▷ Reverzibilní pravé tahy
14:    for každý  $G^{R_i L_j} \in G^{R_i L}$  do
15:      if  $G^{R_i L_j} \geq G$  then ▷  $G^{R_i L_j} \geq G$ 
16:        Nahraď  $G^{R_i}$  všemi  $G^{R_i L_j R_k}$ 
17:         $changed \leftarrow true$ ; break
18:      end if
19:    end for
20:  end for
21:   $G^L \leftarrow \{G^{L_i} : \nexists j \neq i, G^{L_j} \geq G^{L_i}\}$  ▷ Odstraň dominované levé tahy
22:   $G^R \leftarrow \{G^{R_i} : \nexists j \neq i, G^{R_j} \leq G^{R_i}\}$  ▷ Odstraň dominované pravé tahy
23:  if  $G^L$  nebo  $G^R$  se změnilo then  $changed \leftarrow true$ 
24:  end if
25: until  $\neg changed$ 
26:  $K \leftarrow \text{intern}(G)$ 
27:  $\text{canon\_cache}[G] \leftarrow K$  return  $K$ 
```

Kanonizace začíná obcházením reverzibilních tahů. Levý hráč chce hru maximalizovat a pravý ji minimalizovat, takže pokud levý udělá tah do G^{L_i} , je racionální, aby pravý odpověděl na tento tah tahem do pozice, která jeho hodnotu minimalizuje, tedy když $G \geq G^{L_i R_j}$. Po kanonizaci se výsledek předá tabulce `game_intern_cache`, která vrátí existující identický uzel nebo vloží nový.

7.6 Rozdělení Hotpotche na nezávislé komponenty

Součet her, formálně definovaný v sekci 4.3, umožňuje rozdělit pozici na nezávislé komponenty, každou vyřešit zvlášť a výsledné hodnoty sečíst. Dvě pozice v Hotpotchi jsou nezávislé, pokud nesdílejí žádný vrchol kromě případného společného kořene (země). Tah v jedné komponentě tedy nijak neovlivní druhou.

Rozdělení se provádí na grafu živých hran. Smyčky přímo na zemi, tedy hrany, kde $u = v = 0$, se zpracují zvlášť, protože netvoří běžnou komponentu s vrcholem nad zemí. Zbývající komponenty se naleznou prohledáváním do šířky (BFS) přes vrcholy grafu. Pro každý dosud nenavštívený vrchol se projde celá jeho souvislá část a nasbírají se všechny živé hrany, které do ní patří.

Algorithm 7 Rozdělení pozice na nezávislé komponenty

Require: Graf g , živá maska $live$

Ensure: Seznam masek $[m_1, m_2, \dots, m_k]$ nezávislých komponent

```

1:  $components \leftarrow []$ 
2:  $visited[v] \leftarrow \text{false}$  pro všechny vrcholy  $v$ 
3: for každou živou hranu  $e \in live$  do ▷ Speciální případ: smyčka ze země do země
4:   if  $g.edges[e].u = 0$  and  $g.edges[e].v = 0$  then
5:      $components.append(\{e\})$ 
6:      $live \leftarrow live \setminus \{e\}$ 
7:   end if
8: end for
9: for každý vrchol  $i \geq 1$  do ▷ BFS přes vrcholy 1 až  $|V| - 1$ 
10:  if  $visited[i]$  then continue
11:  end if
12:   $queue[0] \leftarrow i$ ;  $head \leftarrow 0$ ;  $tail \leftarrow 1$ ;  $visited[i] \leftarrow \text{true}$ ;  $mask \leftarrow \emptyset$ 
13:  while  $head < tail$  do
14:     $curr \leftarrow queue[head]$ ;  $head \leftarrow head + 1$ 
15:    for každou živou hranu  $e \in live$  do
16:      if  $\neg(g.edges[e].u = curr \text{ or } g.edges[e].v = curr)$  then continue
17:      end if
18:       $mask \leftarrow mask \cup \{e\}$ 
19:       $neighbor \leftarrow$  druhý koncový vrchol hrany  $e$  napojený na  $curr$ 
20:      if  $neighbor \neq 0$  and  $\neg visited[neighbor]$  then
21:         $visited[neighbor] \leftarrow \text{true}$ ;  $queue[tail] \leftarrow neighbor$ ;  $tail \leftarrow tail + 1$ 
22:      end if
23:    end for
24:  end while
25:  if  $mask \neq \emptyset$  then
26:     $components.append(mask)$ ;  $live \leftarrow live \setminus mask$ 
27:  end if
28: end for
29: if  $live \neq \emptyset$  then ▷ Hrany bez cesty do země, nemělo by nastat po cleanup
30:    $components.append(live)$ 
31: end if return  $components$ 

```

Výsledkem je seznam masek, kde každá maska odpovídá jedné nezávislé komponentě. Hodnota celé pozice je poté disjunktivní součet hodnot jednotlivých komponent

$$G = G_1 + G_2 + \dots + G_k,$$

počítaný funkcí `game_add`.

7.7 Porovnávání her: $G \geq H$

Z definice 4.4 platí: $G \geq H$ právě tehdy, když

1. pro každý pravý tah G^R z G platí $\neg(H \geq G^R)$,
2. pro každý levý tah H^L z H platí $\neg(H^L \geq G)$.

Taková definice je dostatečná pro určení jejich uspořádání a je natolik jednoduchá, že režie zásobníku v iterativním algoritmu zpomaluje celý proces. Vzhledem k tomu, že algoritmus používá metodu prohledávání do hloubky (DFS), kde hloubka může být v nejhorším případě počtem hran ve hře, je nepravděpodobné přetečení zásobníku a je lepší využít jednoduchosti rekurzivního algoritmu. Porovnání her je nejdůležitější součástí kanonizace, která zaručuje správný běh výpočtu. Každý takový běh kanonizace využívá porovnání, a tím roste využití této funkce výrazně rychleji než u ostatních pomocných funkcí.

Algorithm 8 Rekurzivní porovnání her $G \geq H$

Require: Hry G a H

Ensure: Pravdivostní hodnota relace $G \geq H$

```

1: if  $G = H$  then return true
2: end if
3: if  $(G, H) \in \text{geq\_cache}$  then return  $\text{geq\_cache}[(G, H)]$ 
4: end if
5: for každý pravý tah  $G^{R_i} \in G^R$  do
6:   if  $H \geq G^{R_i}$  then                                     ▷ Porušení podmínky  $\neg(H \geq G^{R_i})$ 
7:      $\text{geq\_cache}[(G, H)] \leftarrow \text{false}$ 
8:     return false
9:   end if
10: end for
11: for každý levý tah  $H^{L_i} \in H^L$  do
12:   if  $H^{L_i} \geq G$  then                                     ▷ Porušení podmínky  $\neg(H^{L_i} \geq G)$ 
13:      $\text{geq\_cache}[(G, H)] \leftarrow \text{false}$ 
14:     return false
15:   end if
16: end for
17:  $\text{geq\_cache}[(G, H)] \leftarrow \text{true}$  return true

```

Kapitola 8

Zobecnění řešice krátkých her pro ostatní hry

V minulé kapitole byl popsán explicitní algoritmus Hackenbushe. Lze si ale všimnout, že tento algoritmus lze zobecnit pro řešení jakékoliv krátké hry pouhou implementací několika funkcí. Správnou implementací těchto funkcí lze naimplementovanou hru vyřešit bez další úpravy kódu. Byla přidána i dodatečná knihovna Python, kterou lze použít při sestavování rozhraní dané hry. Tento přístup je tedy modulární.

Pro sestavení následujícího modulárního algoritmu byl postupně upraven rekurzivní algoritmus převodu Hotpotche na krátkou hru. Tento postup je ukázán následovně.

V sekci 7.1 je krátká věta o tom, že kombinatorická hra sestavená jako krátká hra (typu `*Game`) *neobsahuje* žádnou informaci o vnitřní struktuře dané hry. Tím je každá část kódu, která používá pouze tento typ, už modulární a nepotřebuje žádné dodatečné úpravy. Tím zbývají všechny části, které používají vnitřní strukturu Hotpotche (resp. Hackenbushe), tedy pracující s typem `BaseGraph`.

Problém je tedy takový: vytvořit rozhraní funkcí, které si specifická kombinatorická hra implementuje, a funkce pro převod na krátkou hru tyto funkce použije.

Pro přehlednost je zde znovu přepsán celý rekurzivní algoritmus převodu na krátkou hru z 5 a ten bude postupně upraven, než se dostanu na finální verzi.

8.1 Obecnější převod surové hry na hru krátkou

Algorithm 9 Rekurzivní výpočet komponenty jako krátké hry

Require: Graf g , maska živých hran $live_mask$

Ensure: Ukazatel na krátkou hru G

```
1: if  $(g, live\_mask) \in position\_cache$  then
2:   return  $position\_cache[g, live\_mask]$ 
3: end if
4: Inicializuj pole  $L \leftarrow [], R \leftarrow []$ 
5: for každou živou hranu  $e \in live\_mask$  do
6:    $child\_mask \leftarrow cleanup(g, live\_mask \setminus \{e\})$ 
7:    $child \leftarrow solve\_component(g, child\_mask)$ 
8:    $c \leftarrow g.edges[e].color$ 
9:   if  $c = \text{modrá}$  nebo  $c = \text{zelená}$  then
10:     $L.append(child)$ 
11:   end if
12:   if  $c = \text{červená}$  nebo  $c = \text{zelená}$  then
13:     $R.append(child)$ 
14:   end if
15: end for
16:  $G \leftarrow game\_make(L, R)$ 
17:  $G \leftarrow game\_canonicalize(G)$ 
18:  $position\_cache[g, live\_mask] \leftarrow G$ 
19: return  $G$ 
```

Jak už bylo popsáno, řádky pracující s typem `*Game` jsou již modulární. Tím zbývají řádky pracující s typem `BaseGraph` a `edge_mask_t`. Ostatní řádky jsou pro přehlednost méně zvýrazněné.

Zobecnění vykonání tahu

Nejllepší způsob je ihned přejít na nekompatibilitu na řádcích 6 a 7. Co tyto řádky v principu dělají? Funkci `cleanup` jsou předány argumenty samotné hry g a masky po zničení hrany na indexu e . Dá se tedy říct, že tyto řádky zodpovídají za vykonání tahu a tato nová pozice je vyjádřena maskou `live_mask`. Obecná funkce by neměla řešit, jak takový tah vypadá, a co se tímto tahem získá. Je proto vhodné definovat funkci, která tento tah vykoná.

Oproti explicitnímu algoritmu je zde ale jeden významný rozdíl. Při vykonání tahu v Hotpotchi (resp. v Hackenbushi) je graf surové hry vždy stejný pro oba hráče. Tento proces ale neplatí obecně. Například ve hře *domineering* (více v sekci 8.2) tah levého hráče mění hrací plochu jinak, než kdyby na stejné místo zahrál pravý hráč. Rozhraní proto bude mít rozdílnou funkci táhnutí pro levého a pravého hráče.

Další otázkou je, co by těmito funkcím bylo předáváno. Taková funkce bude určitě požadovat samotný popis hry. V explicitním algoritmu se jednalo o typ `BaseGraph` (resp. `*BaseGraph`), protože se ale jedná o obecnou funkci, její typ bude ukazatel na void (tedy `void *`) a pomocí klíčového slova `typedef` přejmenován na typ `RawGame_t`. Implementace funkce si ji jen přetypuje.

V sekci 6.1 bylo vysvětleno, proč je použita oddělená proměnná typu `edge_mask_t`. Bylo vysvětleno, že takový tah hry velmi často mění pouze malou část celého popisu hry. V případě Hotpotche se jedná o masku `live_mask`, která popisuje, které hrany jsou zničeny. Pokud by tato dynamická informace byla obsažena ve statické části surové hry `RawGame_t`, vytvářely by se během běhu zbytečné kopie této surové hry. Dávalo by tedy smysl přidat další obecný typ `Position_t` typu `void *`. Posledním argumentem bude index možného tahu `e`. Návrátový typ bude znovu `Position_t`, vyjadřující popis hry po tahu. Tímto způsobem jsou získány následující funkce:

```
1 Position_t do_move_left(RawGame_t raw_game, Position_t position, int e);
2 Position_t do_move_right(RawGame_t raw_game, Position_t position, int e);
```

Výpis 8.1: Funkce pro tah levého a pravého hráče

Zobecnění dotazu kdy vykonat tah

Řádky 8, 9 a 12 vyjadřují dotaz na pravidla hry, zda je možné v dané pozici táhnout. Podobně jako předtím se takové dotazy rozdělí na dvě rozdílné funkce, protože například dotaz na červenou hranu Hackenbushe bude mít jinou odpověď pro každého hráče. Argumentem funkcí bude určitě popis surové hry typu `RawGame_t`, a pokud se vezme znovu příklad z Hotpotche, již zničenou hranu nejde zničit, takže funkcím musí být předána i informace o aktuální pozici typu `Position_t`. Dalším argumentem bude i index možného tahu `e`. Po návratové hodnotě požadují pouze booleovskou odpověď, stačí tedy `int`. Tímto způsobem jsou získány další funkce:

```
1 int can_left_move(RawGame_t raw_game, Position_t position, int e);
2 int can_right_move(RawGame_t raw_game, Position_t position, int e);
```

Výpis 8.2: Funkce pro dotazy, zda v dané pozici může levý nebo pravý hráč táhnout

Zobecnění hashování pozice

Problém na řádcích 1, 2 a 18 je následující. Obecnému algoritmu není jasné, jak takovou hru zahashovat, a tím by nebylo možné použít memoizaci. Zde záleží na rozhodnutí, zda by takový hashovací algoritmus měl použít i informaci o surové hře. Nepoužití surové hry při hashování by znamenalo, že nelze použít data z hashovací tabulky sestavené při předchozím převodu. Jako příklad lze vzít Hotpotch, kde maska vyjadřující informaci o tom, zda je hrana zničená, *neobsahuje* informaci o barvě. Tím mohou mít rozdílné grafy stejnou masku, a nelze tedy použít stejnou tabulku pro různé grafy. Zde jsem si vybral možnost hashování spolu s popisem surové hry.

Pochopitelně dalším argumentem bude pozice typu `Position_t` a index hashovaného tahu `e`. Tato hashovací funkce se provede přes všechny dostupné tahy (zjištěné pomocí funkcí `can_left_move` a `can_right_move`) a zároveň je třeba mít způsob, jak získat počet všech tahů na řádku 5. Je proto třeba implementovat další funkci pro získání této informace. To přidá další funkce:

```
1 uint64_t hash_raw_game_position(RawGame_t raw_game, Position_t position, int e);
2 int num_moves(RawGame_t raw_game);
```

Výpis 8.3: Funkce pro získání počtu všech tahů a hashovací funkce pozice

Zobecnění rozdělení pozice na komponenty

Posledním méně výrazným problémem je rozdělení surové hry na její nezávislé komponenty. Jak bylo popsáno v sekci 7.6, úkolem je rozdělit hru s aktuální pozicí na menší nezávislé pozice, které se sečtou. Zde není velký rozdíl oproti předešlému způsobu, až na to, že implementovaná funkce si musí alokovat vlastní pole pozic, protože obecný řešič velikost tohoto pole nezná. Tím je získána poslední funkce rozhraní:

```
1 int get_independent_components(RawGame_t raw_game, Position_t position,  
2                               Position_t *sub_masks[]);
```

Výpis 8.4: Funkce pro získání pole nezávislých pozic

Výsledný algoritmus vypadá následovně

Algorithm 10 Obecnější rekurzivní převod surové hry na krátkou hru

Require: Surová hra *raw_game*, pozice *position*

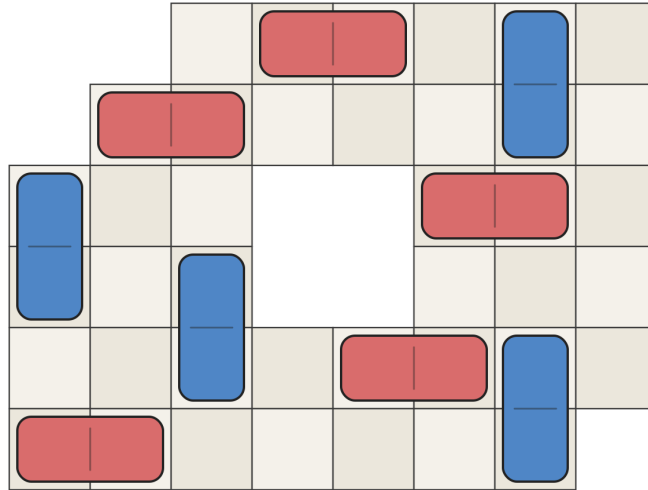
Ensure: Ukazatel na krátkou hru *G*

```
1: if (raw_game, position) ∈ position_cache then  
2:   return position_cache[raw_game, position]  
3: end if  
4: Inicializuj pole  $L \leftarrow []$ ,  $R \leftarrow []$   
5: for  $e = 0, 1, \dots, \text{num\_moves}(\text{raw\_game}) - 1$  do  
6:   if can_left_move(raw_game, position, e) then  
7:     child_position ← do_move_left(raw_game, position, e)  
8:     child ← solve_component(raw_game, child_position)  
9:     L.append(child)  
10:  end if  
11:  if can_right_move(raw_game, position, e) then  
12:    child_position ← do_move_right(raw_game, position, e)  
13:    child ← solve_component(raw_game, child_position)  
14:    R.append(child)  
15:  end if  
16: end for  
17:  $G \leftarrow \text{game\_make}(L, R)$   
18:  $G \leftarrow \text{game\_canonicalize}(G)$   
19: position_cache[raw_game, position] ← G  
20: return G
```

Algoritmus už nepracuje s konkrétní strukturou Hackenbushe. Pro každý index tahu se pouze ptá, zda je daný tah povolený pro levého nebo pravého hráče. Pokud ano, příslušná funkce `do_move_left` nebo `do_move_right` vytvoří novou pozici. Ta se rekurzivně převede na krátkou hru a vloží se mezi levé nebo pravé možnosti.

Tímto způsobem zůstává samotná konstrukce hry $\{L \mid R\}$ stejná pro všechny krátké hry. Rozdíly mezi konkrétními hrami jsou přesunuty do funkcí rozhraní: `can_left_move`, `can_right_move`, `do_move_left`, `do_move_right`, `hash_raw_game_position`, `num_moves` a `get_independent_components`.

8.2 Domineering



Obrázek 8.1: Ukázka nepravidelné pozice ve hře Domineering.

Kombinatorická hra Domineering se obvykle hraje na obdélníkové mřížce, například na desce velikosti $m \times n$, ale obecně může mít rozložení jakékoli. Oba hráči pokládají na prázdná políčka dominové kameny velikosti 1×2 . Levý hráč pokládá kameny pouze svisle, zatímco pravý hráč je pokládá pouze vodorovně. Kámen nesmí překrývat již obsazené políčko a musí celý ležet uvnitř desky. Podle normální konvence prohrává hráč, který již nemůže vykonat žádný tah.

Domineering splňuje podmínky krátké hry. Počet políček na desce je konečný a každý tah obsadí právě dvě dosud volná políčka. Žádný tah tedy nemůže vrátit hru do dřívější pozice a každá partie po konečně mnoha tazích skončí.

Základní popis implementace hry domineering

V implementaci je surová hra reprezentována rozměry hrací desky. Samotná pozice je uložena jako bitová maska obsazených políček. Bit na pozici $i = y \cdot w + x$ určuje, zda je políčko se souřadnicemi (x, y) obsazené.

Struktury použité pro reprezentaci hry jsou následující:

```
1 // src/short-games/C/games/domineering/domineering.h
2 typedef struct {
3     uint8_t width;
4     uint8_t height;
5 } DomineeringBoard;
6
7 typedef struct {
8     uint64_t occupied_mask;
9 } DomineeringPosition;
```

Výpis 8.5: Reprezentace hry Domineering.

Převod surové hry Hackenbushe na krátkou hru už existoval a nedává tedy smysl testovat tento nový způsob na stejné hře. Pro otestování, zda tento nový algoritmus funguje, a ukázkou funkcionality knihovny byly vybrány dvě jednoduché hry: hra domineering 8.2 a hra Toad-and-Frogs [1, str. 12]. Tyto hry implementovaly právě funkce ze sekce 8.1. Zároveň bylo pro tyto hry vytvořeno jednoduché textové rozhraní implementované v jazyce Python.

8.3 Struktura souborů

Práce obsahuje dvě různé metody počítání. První metoda používá na výpočet pozice pouze dyadická čísla z kapitoly 5. Druhá metoda používá strukturu krátkých her.

Soubory pro metodu s dyadickými čísly

Všechny soubory pro tuto metodu se nacházejí v adresáři `src/dyadics`, který se dělí na část s knihovnou (adresář `src/dyadics/C`) a část s grafickým rozhráním (adresář `src/dyadics/python`). Knihovna je napsaná v jazyce C a obsahuje 4 překladové jednotky

1. `raw_game.c` a `raw_game.h`,
2. `dyadics.c` a `dyadics.h`,
3. `hash_game.c` a `hash_game.h`
4. `utils.c` a `utils.h`,

Překladová jednotka `raw_game` obsahuje definici a implementaci funkcí, které jsou zodpovědné za zpracování surové hry. Nachází se zde například definice struktury `BaseGraph` a implementace funkce `cleanup_position`, která vrací pozici po zničení hrany.

Jednotka `dyadics` obsahuje funkce zodpovědné za výpočet dyadických čísel. Nachází se zde definice struktury dyadických čísel a implementace algoritmů 1, 2 a 3.

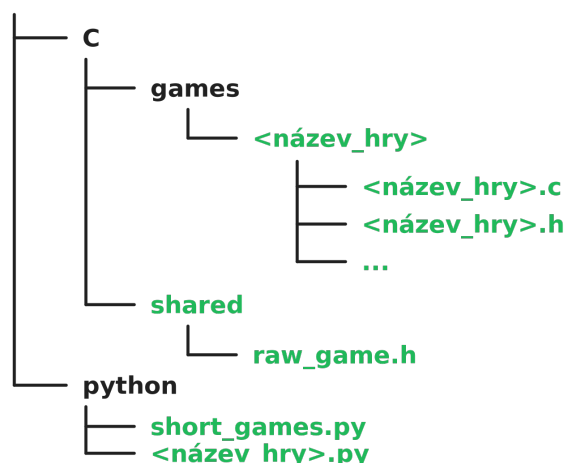
Jednotka `hash_game` obsahuje implementaci funkcí pro hashování pozice a jednotka `utils` obsahuje pomocné funkce jako `floor` a `ceil`.

Soubory pro metodu s krátkými hrami

Všechny soubory této metody se nacházejí v adresáři `src/short-games`, který se podobně dělí na část s knihovnou (adresář `src/short-games/C`) a část s grafickým rozhráním Hotpotche (adresář `src/short-games/python`). Knihovna navíc obsahuje implementaci dalších dvou kombinatorických her, pro které bylo vytvořeno textové rozhraní v Pythonu — soubory `domineering.py`, vycházející ze hry Domineering, a `toads_and_frogs.py`, vycházející ze hry Toads and Frogs, představené v [1, str. 12]. Další kombinatorické hry byly přidány, abych si potvrdil, že takový způsob řešení v principu funguje, a zároveň ukázal, jak může být tato knihovna dále použita. Grafické rozhraní Hotpotche obsahuje mimo jiné i základní kalkulačku krátkých her.

Nové kombinatorické hry se přidávají následovně. V adresáři `src/short-games/C/shared` se nachází soubor `raw_game.h`, jehož funkce musí daná hra implementovat. Jedná se právě o funkce popsané v kapitole 8.1. Implementace tohoto souboru společně s dalšími soubory dané hry se přidává do adresáře `src/short-games/C/games/<název_hry>`.

Důležité je říct, že obecný řešič interaguje s implementovanou hrou pouze prostřednictvím souboru `raw_game.h`. Všechny společné hlavičkové soubory implementované hry a obecného řešiče, včetně souboru `raw_game.h`, se nacházejí v adresáři `src/short-games/C/shared`. V tomto adresáři lze dále najít například `stack.h` nebo `darray.h`.



Obrázek 8.2: Stromová struktura adresářů a souborů se zeleně vyznačenými částmi, se kterými nově implementovaná kombinatorická hra interaguje.

Označení `<název_hry>` představuje zástupný název konkrétní implementované hry. V adresáři `games` jsou již naimplementovány tři hry s adresáři `hotpotch`, `domineering` a `toad_and_frogs`. Jednotné pojmenování *není* požadováno, ale je doporučeno. Program se sestavuje pomocí programu `make` a pokud jsou názvy adresářů a souborů jednotné a na správném místě, potom není nutné samostatně sestavovat danou hru se zbytkem programu, a modul `short_games.py` najde sestavenou knihovnu automaticky.

Nejdůležitějšími jednotkami této knihovny jsou

1. `short_game.c` a `short_game.h`,
2. `singletons.c` a `singletons.h`,
3. `position_cache.c` a `position_cache.h`,
4. `game_canon_cache.c` a `game_canon_cache.h`,
5. `game_intern_cache.c` a `game_intern_cache.h`,
6. `game_operations_cache.c` a `game_operations_cache.h`

Hlavní jednotkou je `short_game`, která obsahuje implementaci nejdůležitějších operací, tedy *porovnání*, *sčítání*, *kanonizaci* a hlavně funkci pro převod surové hry na krátkou hru.

Jednotka `singletons` obsahuje odkazy na důležité krátké hry, jako je hodnota 1, *, ↑ a další. Zároveň se zde nachází i některé pomocné funkce, například funkce pro převod krátké hry na znakovou podobu, nebo funkce pro negaci hry. Tedy praktické funkce, které v principu nejsou potřebné.

Jednotka `position_cache` se stará o memoizaci surové hry. Důvodem odlišného pojmenování oproti ostatním tabulkám je, že tato tabulka pracuje se surovou hrou a je tedy potřeba se o takovou tabulku postarat více než o ostatní, aby byl program nadále modulární.

Jednotky `game_canon_cache`, `game_intern_cache` a `game_operations_cache` využívají typ `Game *` a starají se o standardní memoizační techniky. Tabulka `game_operations_cache` obsahuje dvě různé tabulky, tabulku pro operaci sčítání a tabulku pro operaci porovnání.

Grafické rozhraní Hotpotche

Grafické rozhraní Hotpotche obsahuje několik rozšíření, která jsou relevantní k obsahu této práce nebo byla zadána vedoucím práce.

1. Ukládání a načítání grafů her:

Prvním rozšířením je možnost ukládat a otevírat grafy her. Všechny použité grafy jsou uloženy v rámci kódu, aby si je čtenáři mohli vyzkoušet. Graf hry je uložen ve formátu JSON pomocí funkcí v souboru `hb_io.py`.

2. Analýza pozice:

Analýza pozice graficky znázorní dominované a reverzibilní tahy hráče na řadě. Po zapnutí analýzy obsahuje každá hrana jednu z těchto možností.

- (a) $\langle i \rangle$ značí index dané hrany.
- (b) $\langle i_1 \rangle R \text{ by } \langle i_2 \rangle$ značí, že hrana s indexem $\langle i_1 \rangle$ je reverzibilní tahem s indexem $\langle i_2 \rangle$.
- (c) $\langle i_1 \rangle D \text{ by } \langle i_2 \rangle$ značí, že hrana s indexem $\langle i_1 \rangle$ je dominována tahem s indexem $\langle i_2 \rangle$.

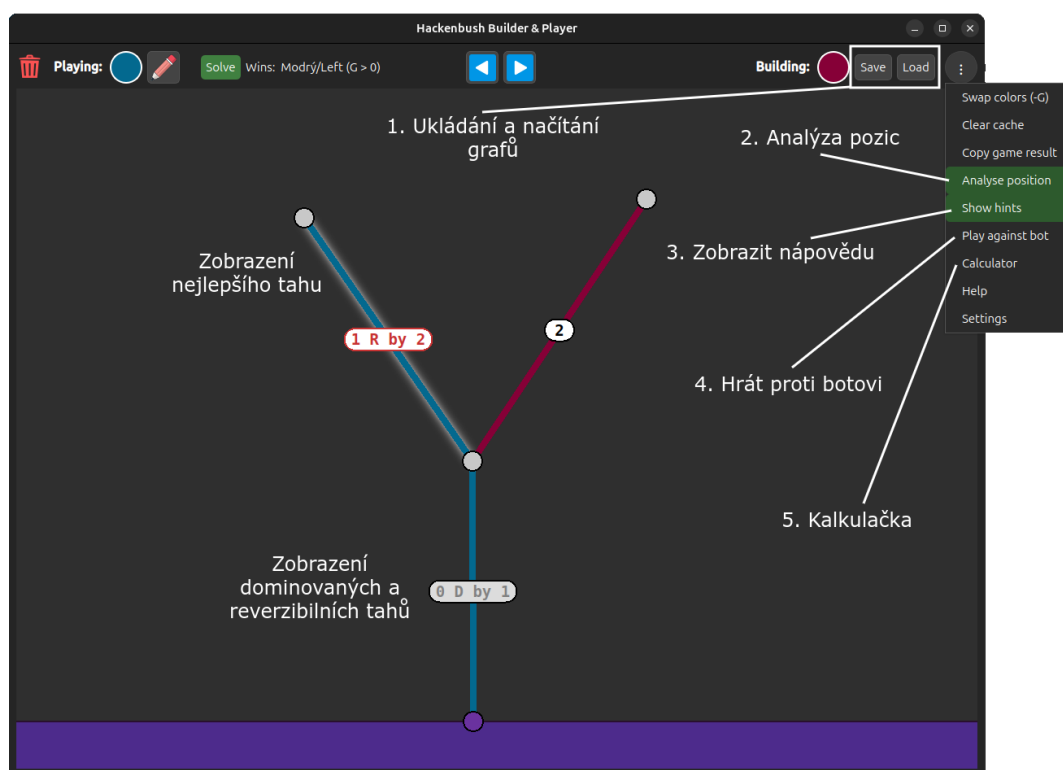
Tato grafická analýza pozice je již použita v obrázcích 4.4 a 4.5. Relevantní metody tohoto rozšíření se nacházejí ve třídě `EduBubble`.

3. Zobrazit nápovědu:

Zobrazit nápovědu (anglicky Show hints) graficky znázorní nejlepší tahy dané pozice, které jsou vyznačeny bílou auroou kolem příslušné hrany. Po najetí na libovolnou hranu je navíc zobrazena hodnota hry po zničení dané hrany.

4. Hrát proti botovi:

Barva hráče je aktuální barva hráče na tahu. Po provedení tahu hráčem bot automaticky zahraje jeden z nejlepších dostupných tahů.



Obrázek 8.3: Ukázka grafického rozhraní Hotpotche s vysvětlivkami daných rozšíření.

5. Kalkulačka:

Kalkulačka je komplexnějším rozšířením této práce. Podporuje základní algebraické operace nad krátkými hrami, společně s negací krátké hry, kanonizací krátké hry a dotazem, zda je daná krátká hra infinitezimální. Motivací k její implementaci byla potřeba hledat vzory v různých součtech a formách krátkých her při vývoji funkce pro převod krátké hry na znakovou podobu.

Calculator ← Back

Syntax: 0 1 -2 -3/4 * *3 ↑ ↓ 2↑ 2^ 3↓ 3v ↑* 1* {0,*|1}

UNARY OPERATION

Canonization =

Result: ☐ raw

BINARY OPERATION

+ ↓ =

Result: ☐ raw

Note: raw output formats only defined number, which is 0 = {|}.

Obrázek 8.4: Ukázka grafického rozhraní kalkulačky krátkých her.

Kapitola 9

Porovnání přístupů a další rozšíření

Práce implementuje dva různé způsoby vyhodnocování pozic Hackenbushe. První způsob pracuje s dyadickými čísly a je určen pouze pro Červeno-Modrý Hackenbush. Druhý způsob používá obecnou strukturu krátkých her a zvládne i Hotpotch, tedy variantu se zelenými hranami.

Porovnání obou přístupů

Přístup přes dyadická čísla má dvě hlavní výhody. Hodnota pozice je přímo číslo, tedy výsledek lze přímo porovnat s nulou bez dalšího zpracování. Odstraňování dominovaných tahů je triviální, protože dyadická čísla jsou lineárně uspořádaná a maximum levých možností, resp. minimum pravých možností existuje vždy. Zřejmou nevýhodou je omezení pouze na Hackenbush. V souboru `src/short-games/python/scripts/test_hackenbush.py` se nachází jednoduché otestování obou přístupů, ze kterého je jasné, že metoda pomocí dyadických čísel je rychlejší. Díky své jednoduchosti je tato metoda vhodná také pro edukativní účely.

Přístup pomocí krátkých her je obecnější. Funguje pro libovolnou kombinatorickou hru, která implementuje rozhraní popsané v kapitole 8. Cena za tuto obecnost je vyšší paměťová a výpočetní náročnost. Výhodou tohoto přístupu je možná jednoduchá rozšířitelnost a práce s krátkými hrami. Dále je možné prázdňě implementovat kombinatorickou hru a využít pouze třídu Python `ShortGameSolver`, která se nachází v souboru `short_games.py`, jako *modul Python krátkých her*.

Tabulka 9.1 shrnuje hlavní rozdíly.

Tabulka 9.1: Srovnání obou přístupů.

	Dyadická čísla	Krátké hry
Podporované kombinatorické hry	Hackenbush	libovolná
Typ výsledku	$\frac{m}{2^n}$	libovolná krátká hra
Paměťová náročnost	nižší	vyšší
Časová náročnost	nižší	vyšší
Obecnost rozhraní	specifické	modulární

Možnosti rozšíření

Nejpřímější rozšíření je upravit metodu využívající dyadická čísla tak, aby byla podobně modulární jako obecný řešič krátkých her. Aktuálně je tato metoda úzce navázána na datovou reprezentaci Hackenbushe. Princip výpočtu pomocí dyadických čísel by se přitom dal použít i pro jiné kombinatorické hry, jejichž hodnoty jsou vždy čísla. Bylo by tedy možné vytvořit obdobné rozhraní jako v kapitole 8, ale s návratovou hodnotou typu `Dyadic` místo `Game *`.

Další možností je přidání dalších kombinatorických her do obecného řešiče krátkých her. Knihovna už obsahuje rozhraní, které umožňuje přidat novou hru implementací několika funkcí.

U grafického rozhraní by byla možná přímá možnost exportu aktuální pozice do obrázku. Funkce pro vytváření obrázků už v aplikaci existuje, ale aktuálně je použita pouze pro generování obrázků do této práce.

Kalkulačku krátkých her by šlo rozšířit o obecnější syntaktický analyzátor výrazů. Nynější verze podporuje základní práci s krátkými hrami, ale neumožňuje pohodlně zapisovat složitější výrazy, jako například součty, rozdíly nebo vnořené operace v jednom řádku.

Samostatným problémem zůstává převod krátké hry na co nejjednodušší znakovou podobu. Implementace nyní rozpoznává již komplexní speciální hodnoty, například $D+n \uparrow$ kde $D \in \mathbb{D}$, $n \in \mathbb{N}$ nebo $n*$ kde $n \in \mathbb{N}$, ale obecný převod by musel rozpoznávat širší třídu hodnot.

Další možností je rozdělit knihovnu na dvě nezávislé části. První část by obsahovala pouze práci s krátkými hrami. Druhá část by obsahovala samotný řešič kombinatorických her. Toto rozdělení by zjednodušilo použití knihovny v případech, kdy uživatel nechce řešit konkrétní hru, ale chce pouze pracovat s krátkými hrami jako algebraickými objekty.

Kapitola 10

Závěr

Cílem práce bylo navrhnout a implementovat řešič pro Hotpotch, který dokáže převést zadanou pozici na její hodnotu v kombinatorické teorii her. Práce nejprve představila potřebný matematický aparát: kombinatorické hry, Hackenbush, Hotpotch, krátké hry, kanonickou formu a dyadická čísla.

V implementační části byly vytvořeny dva rozdílné způsoby řešení. První metoda využívá toho, že hodnoty pozic Hackenbushe lze vyjádřit dyadickými čísly. Díky tomu není nutné uchovávat celý strom krátké hry a stačí pracovat pouze s hodnotou ve tvaru

$$\frac{m}{2^n}.$$

Tato metoda je jednodušší a rychlejší, ale je použitelná pouze pro číselný případ.

Druhá metoda převádí surovou pozici na obecnou krátkou hru. Tato reprezentace je nutná pro Hotpotch, protože zelené hrany vytvářejí hodnoty, které obecně nejsou dyadickými čísly. Implementace proto obsahuje operace nad krátkými hrami, zejména porovnávání, sčítání, negaci a kanonizaci.

Obecný řešič byl následně zobecněn tak, aby nebyl pevně svázán pouze s Hackenbushem a Hotpotchem. Konkrétní hra musí implementovat pouze funkce pro zjištění možných tahů, provedení tahu, hashování pozice a rozdělení pozice na nezávislé komponenty. Tento přístup byl ověřen na hrách Domineering a Toads and Frogs.

Součástí práce je také grafické rozhraní pro Hackenbush a Hotpotch. Rozhraní umožňuje vytvářet a načítat pozice, zobrazovat hodnotu pozice, zvýrazňovat doporučené tahy a graficky ukazovat dominované a reverzibilní tahy. Dále byla vytvořena jednoduchá kalkulačka krátkých her, která využívá stejnou knihovnu jako solver.

Výsledkem práce je tedy funkční implementace dvou přístupů k řešení Hackenbushe. Metoda využívající dyadická čísla ukazuje jednodušší a rychlejší řešení pro Hackenbush. Metoda využívající krátké hry je obecnější a umožňuje hlubší analýzu kombinatorických her.

Literatura

- [1] BERLEKAMP, E. R.; CONWAY, J. H. a GUY, R. K. *Winning Ways for Your Mathematical Plays*. Second. Wellesley, Massachusetts: A K Peters, 2001. ISBN 1-56881-130-6.
- [2] BOUTON, C. L. Nim, A Game with a Complete Mathematical Theory. *Annals of Mathematics*, 1901–1902, sv. 3, 1/4, s. 35–39. Second Series. Dostupné z: <https://www.jstor.org/stable/1967631>.
- [3] CALISTRATE, D. The Reduced Canonical Form of a Game. In: *Games of No Chance*. Mathematical Sciences Research Institute, 1996, sv. 29, s. 409–416. MSRI Publications. Dostupné z: <https://library.slmath.org/books/Book29/files/cali.pdf>.
- [4] CHO, R. *Blue Red Hackenbush Spiders*. San José, California, 2021. Master’s thesis. San José State University. Dostupné z: https://scholarworks.sjsu.edu/etd_theses/5226. [cit. 2026-04-30].
- [5] CONWAY, J. H. *On Numbers and Games*. London: Academic Press, 1976. ISBN 0-12-186350-6.
- [6] ELKIES, N. D. On Numbers and Endgames: Combinatorial Game Theory in Chess Endgames. In: NOWAKOWSKI, R. J., ed. *Games of No Chance*. Cambridge University Press, 1996, sv. 29, s. 135–150. MSRI Publications. Dostupné z: <https://doi.org/10.48550/arXiv.math/9905198>. Available as arXiv:math/9905198.
- [7] KNUTH, D. E. *Surreal Numbers*. Addison-Wesley, 1974. ISBN 0-201-03812-9. Seventeenth printing, April 2006.
- [8] NORVIG, P. Techniques for Automatic Memoization with Applications to Context-Free Parsing. *Computational Linguistics*, 1991, sv. 17, č. 1, s. 91–98. Dostupné z: <https://aclanthology.org/J91-1004/>.
- [9] SCHLEICHER, D. a STOLL, M. *An Introduction to Conway’s Games and Numbers* online. Říjen 2002. Dostupné z: <https://www.cs.cmu.edu/afs/cs/academic/class/15859-s05/www/lecture-notes/comb-games-notes.pdf>. [cit. 2026-04-30].
- [10] SIEGEL, A. N. *Combinatorial Game Theory*. American Mathematical Society, 2013. Graduate Studies in Mathematics. ISBN 978-0-8218-5190-6.

Příloha A

Návod k instalaci a spuštění

Závislosti a překlad

Pro spuštění aplikace je nutné mít nainstalovaný překladač jazyka C s podporou standardu C11 a nástroj `make`. Python musí být ve verzi 3.10 nebo novější.

Nejprve je nutné přeložit sdílenou knihovnu v C. Z kořenového adresáře projektu:

```
1 make lib
```

Výpis A.1: Překlad C knihovny

Virtuální prostředí a Python závislosti

Před spuštěním jakékoliv Python aplikace je doporučeno vytvořit virtuální prostředí:

```
1 python3 -m venv venv  
2 source venv/bin/activate # Linux
```

Výpis A.2: Vytvoření a aktivace virtuálního prostředí

Základní závislosti jsou uvedeny v souboru `requirements.txt` v kořenovém adresáři projektu. Jejich instalace:

```
1 pip install -r requirements.txt
```

Výpis A.3: Instalace závislostí

Spuštění aplikací

Grafické rozhraní Hackenbushe:

```
1 cd src/dyadics/python  
2 python3 hackenbush.py
```

Výpis A.4: Spuštění Hackenbushe

Grafické rozhraní Hotpotche:

```
1 cd src/short-games/python  
2 python3 hotpotch.py
```

Výpis A.5: Spuštění Hotpotche

Textové rozhraní hry Domineering:

```
1 cd src/short-games/python
2 python3 domineering.py
```

Výpis A.6: Spuštění hry Domineering

Textové rozhraní hry Toads and Frogs:

```
1 cd src/short-games/python
2 python3 toads_and_frogs.py
```

Výpis A.7: Spuštění hry Toads and Frogs

Vytváření obrázků pozic

Pro vytváření obrázků pozic slouží skript `src/short-games/python/scripts/create_game_image.py`. Skript umožňuje přesné ovládání výsledného obrázku prostřednictvím následujících proměnných:

- `output_path` — cesta a název výstupního souboru,
- `margin` — velikost okraje kolem ohraničení pozice, pokud je velikost ohraničení určena automaticky,
- `fixedX` — pevná šířka ohraničení; obsah je vystředěn horizontálně,
- `fixedY` — pevná výška ohraničení; obsah je vystředěn vertikálně,
- `finalX` — výsledný obrázek je přizpůsoben na tuto šířku bez ohledu na původní velikost ohraničení,
- `finalY` — výsledný obrázek je přizpůsoben na tuto výšku bez ohledu na původní velikost ohraničení,
- `include_bubbles` — pokud je nastaveno na `True`, jsou v obrázku zobrazeny grafické ukazatele dominovaných a reverzibilních tahů, jak je popsáno v sekci 7.6.

Všechny obrázky pozic použité v této práci se nacházejí ve složce `src/short-games/python/thesis/games`.